ESCOUADE

Audit de sécurité

Escouade 2.2.0, application de bureau : lecture du code, essais en boîte blanche et en boîte noire

Objet	L'état de la sécurité d'Escouade 2.2.0, au 14 novembre 2026
Version auditée	Escouade `2.2.0` (étiquette `v2.2.0`, commit `ab351a00`), publiée le 14 novembre 2026
Date	14 novembre 2026
Nature	Audit interne, mené par un agent d'IA (Claude) pour l'éditeur. Ce n'est pas un audit indépendant.
Constats	18, dont 3 de gravité moyenne, 14 faible, 1 d'information. Aucun critique, aucun élevé.
Diffusion	Publique. Le document ne contient ni secret, ni jeton, ni donnée personnelle.

Chaque constat dit s'il est **démontré** (essai reproductible, livré avec le dépôt), **probable** (lu dans le code, pas déclenché) ou **théorique**. Les preuves citent le code par fichier et ligne, au commit de la version auditée. Les essais, leurs sources et leurs résultats bruts sont dans `docs/security/audit-2026-11/` du dépôt.

Sommaire

1. Résumé exécutif 3
2. Périmètre, version, méthode et limites 5
3. Architecture et frontières de confiance 8
4. Inventaire des flux réseau et des données stockées 9
5. Les constats 13
   - SEC-01 — Plugins tiers : installables sans revue hors du store, et capables de faire lancer un programme — Moyenne — 14
   - SEC-02 — « Projet privé » est une consigne, pas un cloisonnement — Moyenne 15
   - SEC-03 — Sceau « Vérifié par Escouade » : une seule clé, sans mot de passe, et une revue par un agent — Moyenne — Accepté — 16
   - SEC-04 — Dépendances : des avis ouverts hors de l'application, sans délai de correction — Faible 18
   - SEC-05 — Politique de contenu : styles en ligne autorisés, directives manquantes — Faible 20
   - SEC-06 — Lancement décrit par un plugin : un fichier écrit à travers un lien — Faible 21
   - SEC-07 — Le jeton du serveur MCP passe en argument et reste en clair chez les harness — Faible 22
   - SEC-08 — Flux d'un harness : une longueur annoncée suffit à arrêter l'application — Faible 23
   - SEC-09 — Réseau des plugins : trois angles morts — Faible 24
   - SEC-10 — Données au repos : pas de trousseau sous Linux, dossier lisible sous Unix — Faible 25
   - SEC-11 — La sortie d'un plugin n'est pas plafonnée pendant un tour — Faible 26
   - SEC-12 — Fenêtre principale : des droits très larges si un script y entrait — Faible 27
   - SEC-13 — Des variables d'environnement de test restent actives en version publiée — Faible 28
   - SEC-14 — Mises à jour : le proxy est contourné s'il ne répond pas — Faible 29
   - SEC-15 — Liste de révocation des plugins : pas de date de péremption — Faible 30
   - SEC-16 — Site de l'éditeur sans en-têtes de sécurité — Faible 31
   - SEC-17 — Serveurs locaux : un programme du poste peut les ralentir — Faible 32
   - SEC-18 — Observations diverses — Info 33
6. Les contrôles en place 35

Annexe A. Dépendances Rust : cargo audit et cargo deny 46

Annexe B. Dépendances npm : npm audit 49

Annexe C. Analyse statique et dépendances directes 52

Annexe D. Tests réalisés 56

Annexe E. Guide de durcissement pour un poste d'entreprise 61

Annexe F. Reproduire l'audit 63

1. Résumé exécutif

Ce qu'est Escouade

Escouade est une application de bureau (Windows, macOS, Linux) qui pilote des agents de programmation : Claude Code, Codex, Copilot CLI, Cursor CLI, Gemini CLI et d'autres. Elle lance ces programmes (les « harness »), affiche leurs conversations, organise leur travail en tickets et en worktrees git, et le relie aux hébergeurs de code. Elle n'a pas de serveur à elle : **le code et les conversations restent sur le poste**, et ne le quittent que par les harness, vers leur fournisseur de modèle. Un agent de programmation exécute des commandes avec les droits de l'utilisateur et lit du contenu écrit par des tiers : l'audit regarde ce qu'Escouade ajoute à ce risque, et comment elle tient les garde-fous des harness.

Ce que l'audit conclut

L'audit n'a trouvé dans Escouade 2.2.0 aucune faille critique ni élevée : aucune exécution de code à distance sans action de l'utilisateur, aucune sortie de code ou de conversation par défaut vers l'éditeur, aucun port ouvert sur le réseau.

Il relève 18 constats. Les trois de gravité moyenne tiennent à ce que l'application laisse faire à l'utilisateur, ou à la façon dont l'éditeur travaille, pas à une faille à exploiter :

1. **Plugins tiers** (SEC-01). « N'installer que des versions vérifiées » n'est allumé que pour une installation neuve, et ne juge que le store. Un plugin tiers peut décrire un harness, donc faire lancer un programme du poste hors du bac à sable ; la fenêtre d'installation montre la ligne de commande.
2. **Projets privés** (SEC-02). Marquer un projet « privé » le cache aux agents des autres projets ; côté fichiers, c'est une consigne donnée aux harness, pas un cloisonnement du système. L'interface le dit.
3. **Sceau « Vérifié par Escouade »** (SEC-03). Une seule clé, gardée sans mot de passe sur le poste de l'éditeur, et une revue faite par un agent. L'éditeur connaît ce point et garde ce fonctionnement.

Les quatorze constats faibles sont de la défense en profondeur : des gardes qui tiennent mais dont un bord reste ouvert, ou des plafonds qui manquent. Le dernier constat réunit des observations sans impact direct.

Une réserve : cet audit est interne, mené par un agent d'IA pour l'éditeur (section 2, « Limites »). Il dit l'état d'une version à une date ; il ne vaut pas pour les versions suivantes.

Les constats par gravité

Gravité	Nombre	Constats
Critique	0	—
Élevée	0	—
Moyenne	3	SEC-01 à SEC-03 : plugins tiers, projets privés, sceau des plugins
Faible	14	SEC-04 à SEC-17 : avis de dépendances hors de l'application, politique de contenu, lien dans un dossier d'agent, jeton en argument, lecteurs sans plafond, bords du réseau des plugins, données au repos sous Unix, droits de la fenêtre, variables de test, proxy contourné s'il est en panne, liste de révocation sans péremption, en-têtes du site, serveurs locaux
Info	1	SEC-18 : observations diverses

Ce qui est en place

Vérifié par les essais, et détaillé au chapitre 6 :

- **Réseau.** Rien n'écoute hors de `127.0.0.1`. Le serveur local par lequel les agents pilotent Escouade est éteint par défaut, gardé par un jeton et par le contrôle des en-têtes. L'application n'appelle pas le fournisseur de modèles elle-même : ce sont les harness qui le font.
- **Agents.** Un agent ne répond pas à la demande de permission d'un autre. Le mode sans aucune demande n'existe que sur un réglage éteint par défaut.
- **Plugins.** Ils tournent dans un bac à sable WebAssembly qui applique leurs permissions : réseau, fichiers, programmes, variables d'environnement, secrets. Un dossier chargé « en développement » n'a jamais les droits de l'éditeur. Une mise à jour que personne n'a relue a toujours sa propre fenêtre, avec son avertissement.
- **Fenêtre.** Le contenu des agents s'affiche comme du texte ; la politique de contenu bloque tout script injecté.
- **Secrets et certificats.** Les jetons vont au trousseau du système. Derrière un proxy qui inspecte le TLS, un fichier d'autorités de certification suffit : la vérification des certificats reste active.
- **Chaîne d'approvisionnement.** Mises à jour signées ; avis de sécurité des dépendances contrôlés à chaque changement et chaque nuit, aucun n'est atteignable dans l'application ; actions de la CI figées par empreinte. Les cinq binaires publiés de la 2.2.0 portent une attestation de provenance vérifiée, qui les relie au commit audité.

Recommandations pour un poste d'entreprise

La version. 2.2.0 ou plus récente.

Les réglages, sur chaque poste (détail en annexe E). Une installation mise à jour depuis une version antérieure garde ses anciens choix : vérifier les points 1 à 3.

1. **Mode des agents** : « Demander » ou « Plan », pour chaque compte de harness et pour le Kanban. « Autoriser le mode Bypass » : éteint.
2. **« Les agents peuvent piloter Escouade » : éteint** (le défaut). S'il doit être allumé, laisser la réponse aux permissions des autres agents sur « Personne ».
3. **Plugins** : allumer « N'installer que des versions vérifiées » ; n'installer aucun plugin tiers, ni du store, ni d'un dossier, ni d'un dépôt ; ne charger aucun plugin « en développement ».
4. **Remote Control : éteint** (le défaut). **Statistiques d'usage : refuser. Voix en ligne : ne pas configurer.**
5. **Réseau** : derrière un proxy qui inspecte le TLS, donner le certificat racine de l'entreprise ; ne pas cocher « Ignorer la vérification des certificats ». N'ouvrir au pare-feu que les destinations de l'inventaire (section 4).
6. **Harness** : leur appliquer la politique de l'entreprise sur les fournisseurs de modèles. Ce sont eux, pas Escouade, qui envoient le code lu par l'agent.
7. **Projets privés** : ne pas s'y fier pour séparer des projets qui ne doivent pas se voir. Un poste, ou un compte du système, pour chacun.

2. Périmètre, version, méthode et limites

Périmètre

L'application de bureau telle qu'elle s'installe, en huit parties :

#	Partie	Sections
1	Ce qui sort du poste : toutes les connexions réseau	4.1, 6.1
2	Les secrets : où ils sont, qui les lit, ce qui va aux journaux	4.2, 6.2, SEC-07, SEC-10
3	Les agents : permissions, modes, clôture, hooks, serveur MCP local, Remote Control	6.3, SEC-02, SEC-17
4	Les plugins : hôte WebAssembly, capacités, signatures, révocation, lancement décrit par un plugin	6.4, 6.9, 6.10, SEC-01, SEC-03, SEC-06, SEC-08, SEC-09, SEC-11
5	La chaîne d'approvisionnement : dépendances, CI, signature des binaires, reproductibilité	6.5, 6.11, SEC-04, annexes A et B
6	La webview : politique de contenu, commandes exposées, liens, contenu tiers affiché	6.6, SEC-05, SEC-12
7	Les données sur le disque	4.2, 6.7, SEC-10
8	Le store de plugins et le site, pour ce qui touche la confiance dans un plugin installé	6.8, SEC-03, SEC-15, SEC-16

Hors périmètre : la sécurité des harness eux-mêmes (Claude Code, Codex…) et de leurs fournisseurs, l'hébergement du store et du site au-delà de leurs en-têtes et de leur TLS, les applications mobiles (il n'y en a pas).

Version auditée

- **Escouade 2.2.0** : étiquette `v2.2.0`, commit `ab351a0047b8` du dépôt public de l'éditeur, publiée le 14 novembre 2026. Les preuves de ce rapport citent ce commit, et tous ses essais ont été joués sur lui.
- Les essais en boîte noire ont porté sur un binaire de développement construit à ce commit, sous Windows 11. Un binaire de développement diffère d'un binaire publié sur trois points qui comptent ici : il lit des variables d'environnement de test (adresse du store, du serveur de mises à jour), il n'est pas signé, et les outils de développement de la fenêtre y sont ouverts. Ces écarts sont signalés là où ils jouent.
- Les binaires publiés de cette version ont été contrôlés sans être lancés : signatures et attestation de provenance (section 6.11).

Méthode

1. **Lecture du code** : le cœur (environ 150 000 lignes de Rust hors fichiers de tests), les crates partagées et les plugins de première main (95 000 lignes), l'interface (105 000 lignes de Svelte et de TypeScript), le store (20 000 lignes), les workflows de CI. La lecture a été ciblée sur les frontières de confiance : elle n'est pas exhaustive ligne à ligne.
2. **Outils d'analyse** : `cargo audit` 0.23, `cargo deny` 0.20, `npm audit`, `cargo geiger` 0.13, `semgrep` 1.185 avec 149 règles publiques. Résultats en annexes A, B et C.
3. **Essais en boîte blanche** : 38 tests Rust d'attaque écrits pour l'audit contre les frontières des plugins, des harness et du réseau, dont 7 boucles d'entrées générées (fuzzing, 2 minutes chacune). 467 tests existants du projet, choisis sur les mêmes frontières, ont été rejoués.

4. **Essais en boîte noire** : l'application lancée isolée (dossier de données et profil de test, faux harness), attaquée par 16 scénarios : ports, serveur MCP, contenu tiers, plugin malveillant, mises à jour des plugins, canal de mise à jour. Un plugin malveillant a été écrit pour l'audit.
5. **Contrôle du produit publié** : les fichiers de la version téléchargés depuis sa page, et vérifiés sans être lancés par leurs signatures et leur attestation de provenance.
6. **Lecture réseau du store et du site**, les 12 et 14 novembre 2026 : une douzaine de requêtes en lecture seule (en-têtes, TLS, CORS). Aucune écriture, aucun compte, aucune charge.

L'annexe D liste tous les essais, y compris ceux qui n'ont rien trouvé.

Limites

Ce qui suit n'a **pas** été vérifié, et le lecteur doit le savoir :

- **L'audit est interne.** Il a été mené par un agent d'IA mandaté par l'éditeur, avec accès au code. Il ne remplace pas un test d'intrusion par un tiers indépendant.
- **Les binaires publiés ont été contrôlés sans être lancés** (section 6.11). Seul un binaire de développement construit au même commit a été lancé : un binaire publié ignore les variables par lesquelles les essais remplacent les connexions du poste par des fichiers d'essai (`src-tauri/src/accounts.rs:874-878` , `:926-930`), et il aurait lu celles du poste de l'audit, ce que cet audit s'interdit. Deux choses en tiennent lieu : les règles éprouvées ne dépendent pas du type de construction, sauf mention ; et l'attestation de provenance relie chaque binaire publié au commit audité. La notarisation de l'application macOS est lue dans le journal du job qui l'a faite, et son ticket est présent dans le paquet ; elle n'a pas été rejouée sur un Mac. La reproductibilité des binaires n'est pas établie (section 6.5).
- **Le code n'a pas été lu en entier.** La lecture suit les frontières de confiance du chapitre 3.
- **Seul Windows a été essayé en boîte noire.** Les constats propres à macOS et à Linux (trousseau, permissions de fichiers, bac à sable des harness) viennent de la lecture du code.
- **Les harness réels n'ont pas été lancés.** Les essais utilisent les faux harness du projet. Le comportement de Claude Code face à une règle de refus, par exemple, est supposé conforme à sa documentation ; de même le sens des arguments que l'application retire d'un lancement, et la tenue des refus qu'elle donne à Copilot CLI.
- **Les fichiers de connexion et de clés du poste n'ont pas été lus**, par consigne : trousseau du système, identifiants des harness, dossier des clés de signature de l'éditeur. SEC-03 repose donc sur la documentation et l'outillage du dépôt, et sur une précision de l'éditeur.
- **Les avis de sécurité des dépendances sont cités tels que leurs bases les publient.** Leur exploitation n'a pas été tentée.
- **Aucun service tiers n'a été attaqué** : ni GitHub, ni les fournisseurs de modèles, ni le service de statistiques.
- **Le fuzzing est léger** : des boucles d'entrées mutées, sans instrumentation de couverture. `cargo-fuzz` n'a pas été utilisé (il demande une chaîne d'outils que le poste de l'audit n'a pas).
- La lecture n'étant pas exhaustive, **l'absence de constat sur une partie du code n'est pas une preuve d'absence de faille**.

Échelle de gravité

Gravité	Sens dans ce rapport
Critique	Exécution de code, ou fuite du code de l'utilisateur, sans action de sa part, dans la configuration par défaut
Élevée	Atteinte grave à la confidentialité ou à l'intégrité, exploitable par un attaquant réaliste, avec une condition (position sur le réseau, option allumée)
Moyenne	Affaiblit un garde-fou annoncé, ou demande une condition forte (consentement de l'utilisateur, accès local)

Gravité	Sens dans ce rapport
Faible	Défense en profondeur, impact limité ou condition peu probable
Info	Observation sans impact de sécurité direct

Un score CVSS 3.1 est donné quand il a un sens. Il décrit le pire cas dans les conditions du constat, pas le risque moyen.

3. Architecture et frontières de confiance

Escouade est un seul processus Rust qui porte une fenêtre web locale, et qui lance d'autres programmes. Six frontières séparent ce qui est de confiance de ce qui ne l'est pas.

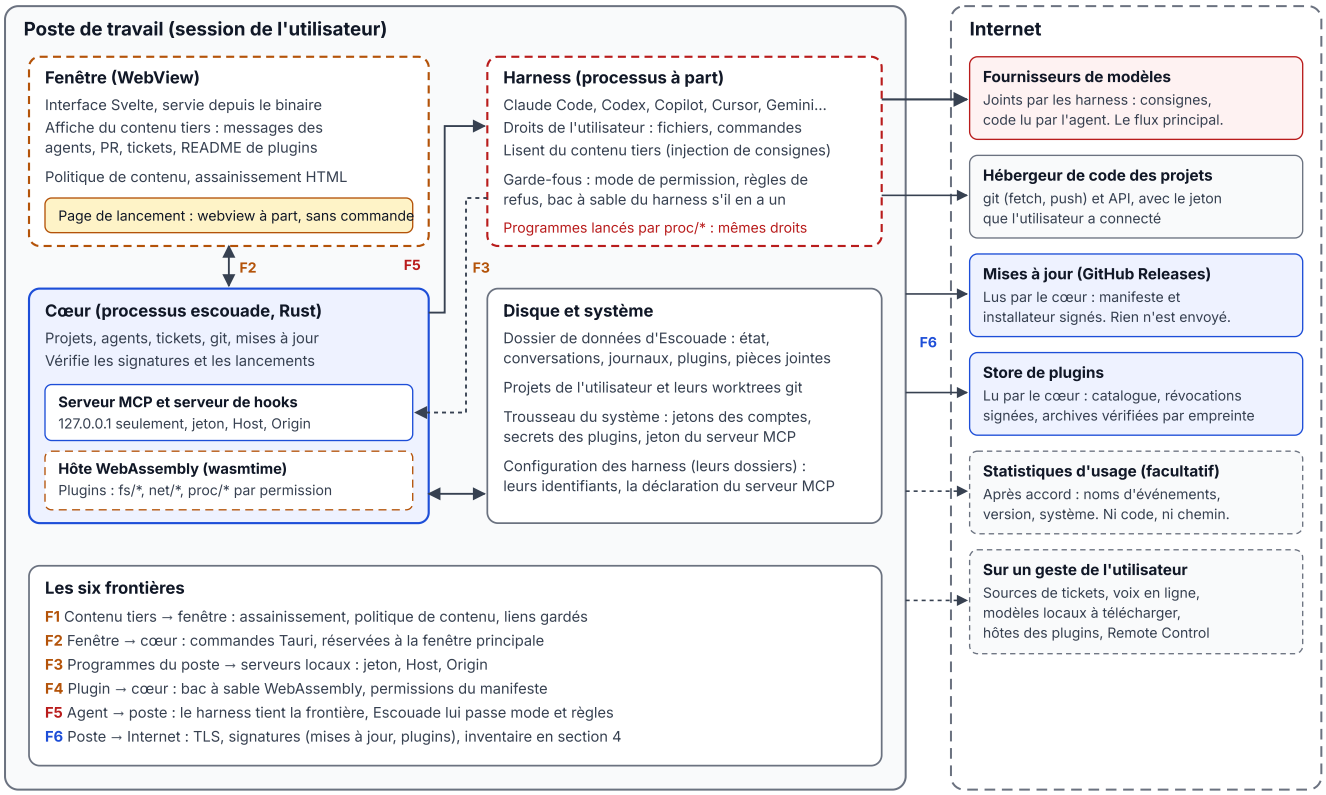


Figure 1. Les composants d'Escouade et ses frontières de confiance. Cadres en pointillés orange ou rouge : ce qui traite ou produit du contenu non fiable. Flèches en pointillés vers Internet : flux facultatifs.

Qui est de confiance. Le cœur et l'interface, livrés et signés par l'éditeur. Les plugins de première main (`dev.escouade.*`), qui ne s'installent que signés.

Qui ne l'est pas. Le contenu que les agents lisent et produisent : un fichier du dépôt, une page web, une description de pull request peuvent porter des consignes qui détournent un agent. Les plugins tiers. Les réponses des services en ligne. Les autres programmes du poste, pour ce qui est des serveurs locaux.

Ce que l'application ne cloisonne pas. Un harness tourne avec les droits de l'utilisateur. Escouade ne le met pas dans un bac à sable du système : elle lui transmet un mode de permission et des règles, et c'est le harness qui les applique. Tout ce que l'utilisateur peut lire sur son poste, un agent détourné peut le lire, dans la limite de ce que son harness lui refuse.

4. Inventaire des flux réseau et des données stockées

4.1 Les connexions réseau

L'inventaire vient du code : toutes les adresses écrites dans les sources Rust et TypeScript hors tests, puis la lecture de chaque appelant. Les essais en boîte noire l'ont recoupé pour ce qui écoute (section 6.1). « Par défaut » veut dire : sans aucun geste de l'utilisateur après l'installation.

#	Destination	Ce qui part	Quand	Par défaut	Comment couper	Preuve
R1	`github.com`, dépôt public de l'éditeur : `…/releases/latest/download/latest.json` (et `beta.json` si les bêtas sont demandées), puis l'installateur	Une requête GET, l'agent utilisateur `Escouade/<version> (<système>; <architecture>)`. Aucun identifiant	8 s après le démarrage, puis toutes les 5 minutes	**Oui**	Aucun réglage ne coupe le contrôle ; « Installer automatiquement » ne règle que le redémarrage. Bloquer au proxy	`src-tauri/tauri.conf.json:74`, `src/lib/updater.ts:13-14`, `src/App.svelte:84`, `crates/escouade-updates/src/net.rs:116-132`
R2	`store.escouade.dev` : `/api/v1/catalog.json`, `/api/v1/revocations.json` et sa signature ; `escouade.dev/revocations.json` si le store ne répond pas	Des requêtes GET sans identifiant	Au démarrage, puis toutes les 6 heures	**Oui**	Aucun réglage. Bloquer au proxy : les plugins installés continuent de tourner	`crates/escouade-plugins/src/store.rs:26-30`, `src-tauri/src/core.rs:2471-2478`
R3	Remotes git de chaque projet (l'hébergeur de code du projet)	`git fetch`, avec les identifiants git de l'utilisateur	Toutes les 5 minutes pour chaque dépôt qui a un remote	**Oui**, dès qu'un projet est ajouté	Aucun réglage repéré	`src-tauri/src/core.rs:751-752`
R4	Anthropic, par Claude Code : le cœur lance un `claude` du compte, sans tour ni session, qui lit ses quotas (`initialize`, puis `get_usage`)	Ce que Claude Code envoie avec sa propre connexion. Le cœur n'appelle pas Anthropic et ne tient pas de jeton : il lit le fichier de connexion de Claude Code pour savoir si un compte est connecté	Au plus toutes les 5 minutes par compte actif, quand aucun de ses agents n'a répondu	**Oui**, si Claude Code est connecté	Aucun réglage dédié ; désactiver le compte	`src-tauri/src/core.rs:10969-11021`, `src-tauri/src/engine/claude/mod.rs:646-658`, `plugins/claude/src/terminal.rs:116-136`, `src-tauri/src/usage.rs:451`
R5	Fournisseurs de modèles des harness (Anthropic, OpenAI, GitHub, Google, Mistral, Cursor…)	Par les harness eux-mêmes : consignes, code lu par l'agent, résultats d'outils	À chaque tour d'un agent	**Oui**, c'est la fonction du produit	Choix du harness et du compte ; politique de l'entreprise sur ces CLI	Les harness sont lancés par `src-tauri/src/plugins/harness.rs:1367-1392`
R6	Mise à jour de Claude Code (`claude update`, serveurs d'Anthropic ou npm)	Ce que la commande de Claude Code envoie	Au démarrage, puis une fois par jour	**Oui**	Réglages › Comptes (réglages avancés) › « Mettre à jour Claude Code automatiquement »	`src-tauri/src/selfupdate.rs:21`, `crates/escouade-model/src/model.rs:751`
R7	Statistiques d'usage : `aptabase.gagnaire.dev` (instance de l'éditeur)	Nom de l'événement, version, système, langue, identifiant d'installation aléatoire. Jamais un nom, un chemin ou un texte	Toutes les minutes s'il y a des événements, et à la fermeture	**Non** : rien tant que l'utilisateur n'a pas dit oui	Réglages › Général › Statistiques d'usage	`src-tauri/src/telemetry.rs:21-22`, `:37`, `:391-393`, `crates/escouade-model/src/model.rs:756`
R8	API des hébergeurs de code : `api.github.com`, `gitlab.com`, `api.bitbucket.org`, `dev.azure.com`, ou l'instance saisie (Gitea, Forgejo, GitHub Enterprise)	Le jeton du compte ; lecture et écriture de pull requests, de CI, de tickets	Quand l'utilisateur a connecté un compte, puis périodiquement	Non	Ne pas connecter de compte, ou le déconnecter	Plugins `plugins/github`, `gitlab`, `bitbucket`, `gitea`, `azure-devops` ; `crates/escouade-plugins/src/wasm.rs:1125-1183`

#	Destination	Ce qui part	Quand	Par défaut	Comment couper	Preuve
R9	Sources de tickets : Jira, Trello, Linear, ClickUp, Azure Boards	Le jeton du compte ; lecture et mise à jour de tickets	Quand un compte est connecté	Non	Ne pas connecter de compte	`src-tauri/src/integrations/mod.rs`, plugins `jira`, `trello`, `linear`, `clickup`
R10	Voix en ligne : `api.openai.com`, `api.groq.com`, `api.deepgram.com`, `api.elevenlabs.io`, `api.mistral.ai`	L'audio du micro ou le texte à lire, la clé d'API saisie	Quand l'utilisateur dicte ou écoute, après avoir configuré un service	Non	Ne pas configurer de service ; la dictée locale n'envoie rien	`crates/escouade-voice/src/online.rs`
R11	`huggingface.co` : modèles de dictée (whisper.cpp), voix (Piper), recherche par le sens	Des requêtes GET	Au clic sur « Télécharger »	Non	Ne pas télécharger	`crates/escouade-voice-local/src/models.rs:153-182`, `crates/escouade-vectors/src/model.rs:49-56` (empreintes SHA-256 épinglées)
R12	Store : archive, module et signature d'un plugin ; `POST /api/v1/installs`	À l'installation : des GET. Le comptage (identifiant et version du plugin, sans identifiant du poste) seulement si les statistiques sont acceptées	Au clic sur « Installer »	Non	Ne pas installer de plugin	`crates/escouade-plugins/src/store.rs:711-840`, `src-tauri/src/commands.rs:486-492`
R13	Hôtes déclarés par un plugin (`permissions.network`)	Ce que le plugin envoie : par exemple un message vers un webhook de messagerie	Selon le plugin	Non	Ne pas installer le plugin, ou lui retirer la permission	`crates/escouade-plugins/src/wasm.rs:1125-1183`
R14	`github.com` : l'outil Javy (compilation des plugins JavaScript)	Une requête GET ; le binaire est vérifié par son SHA-256	Au premier chargement d'un plugin JavaScript depuis un dossier	Non	Ne pas charger de plugin en développement	`crates/escouade-plugin-cli/src/wasm.rs:231-250`
R15	Remote Control : `claude.ai` (Anthropic), par Claude Code	La conversation de l'agent, pilotable depuis le compte claude.ai de l'utilisateur	Quand l'utilisateur l'allume (global, projet ou agent)	Non	Réglages › Agents ; réglages du projet	`crates/escouade-model/src/model.rs:763`, `src-tauri/src/core.rs:4528-4530` ; aucun outil MCP ne l'allume (`src-tauri/src/mcp/tools.rs:226`)
R16	L'hôte d'une image citée dans un message	Une requête GET en https, sans cookie, sans référent, sans agent utilisateur	Au clic sur « Afficher l'image »	Non	Ne pas cliquer	`src-tauri/src/sentfiles.rs:594-625`, `src/lib/markdown.ts:27-46`
R17	Navigateur du système : documentation, communauté, page de soutien	L'ouverture d'un lien	Au clic	Non	—	`src/lib/links.ts:19-30`
R18	Serveur de développement du projet (`localhost`)	La page de l'application que l'utilisateur développe, dans une webview à part	Au lancement d'un test par l'utilisateur	Non	—	`src-tauri/src/devpage.rs:1-15`

Ce qui part par défaut, sans geste : R1 (contrôle de mise à jour), R2 (catalogue du store), puis, dès qu'un projet et un compte existent, R3 (git fetch), R4 (quotas, par Claude Code), R5 (les harness) et R6. Aucun de ces flux ne porte de code vers l'éditeur d'Escouade. Le code de l'application ne nomme aucune adresse d'Anthropic : R4, R5, R6 et R15 sont des connexions de Claude Code.

Le proxy. Réglages › Avancé › Réseau prend une adresse de proxy, une liste d'exclusions, un fichier d'autorités de certification, et l'option « Ignorer la vérification des certificats TLS ». Le proxy s'applique aux requêtes de l'application et aux harness (variables `HTTPS_PROXY`). Deux points à connaître : si le proxy configuré ne répond pas, le contrôle de mise à jour tente une fois une connexion directe (SEC-14) ; l'option sur les certificats, déconseillée, ne vaut que pour une connexion qui passe par le proxy (section 6.1).

Ce qui écoute. Rien sur le réseau. Sur `127.0.0.1` : le serveur MCP (port entre 47000 et 47999, seulement si « Les agents peuvent piloter Escouade » est allumé) et le serveur de hooks (port libre, seulement quand une vue terminal d'un harness est ouverte).

4.2 Les données stockées

Le dossier de données est `~/.escouade` (sous Windows, dans le profil de l'utilisateur). Les essais ont relu les fichiers écrits par l'instance de test à la recherche de jetons : aucun hors des fichiers prévus pour en tenir (annexe D).

Donnée	Emplacement	Contient du code ou des conversations	Protection	Preuve
État : projets, agents, tickets, réglages	`state.json`, `settings.json`	Chemins des projets, titres et textes des tickets. L'adresse du proxy, avec son mot de passe si l'utilisateur l'a écrit dans l'URL	Droits du profil de l'utilisateur	`crates/escouade-model/src/paths.rs:301-325`, `crates/escouade-model/src/model.rs:35`
Conversations des agents	`conversations/`, et les dossiers des harness (`~/.claude`, etc.)	**Oui** : consignes, extraits de code, sorties de commandes	Droits du profil ; en clair	`src-tauri/src/paths.rs` (`conversations`)
Pièces jointes des messages et des tickets	`attachments/`, `tickets/`	**Oui** si l'utilisateur en joint	Droits du profil ; purge réglable	`crates/escouade-model/src/fence.rs:122-124`
Journal de l'application	`app.log`	Chemins, lignes de lancement des harness. Les consignes passées en argument sont remplacées par leur longueur	Droits du profil	`src-tauri/src/plugins/harness.rs:1342-1355`
Journaux des plugins	`plugin-logs/`	Ce que le plugin écrit ; ses secrets y sont masqués	Droits du profil	Essai : annexe D, B-09
Jetons des comptes (hébergeurs, tickets, voix), secrets des plugins, jeton du serveur MCP	Trousseau du système (Gestionnaire d'identification de Windows, Trousseau de macOS). **Sous Linux : un fichier**, lisible par son propriétaire seul	Non	Trousseau ; fichier en mode 0600 sinon (SEC-10)	`crates/escouade-forge/src/integrations/secrets.rs:54-138`, `crates/escouade-model/src/paths.rs:691-713`
Jeton de chaque agent pour le serveur MCP	`mcp/<agent>.json`, effacé à l'arrêt de l'agent	Non	Fichier privé ; en mémoire sinon	`src-tauri/src/mcp/mod.rs:360-393`

Donnée	Emplacement	Contient du code ou des conversations	Protection	Preuve
Déclaration du serveur MCP pour les harness	Fichiers de configuration des harness (par exemple celui de Claude Code)	Non, mais **le jeton y est en clair** (SEC-07)	Droits du profil	`src-tauri/src/engine/claude/mcp.rs:20-30`
Connexion de Claude Code (son jeton OAuth)	Les fichiers de Claude Code : `.credentials.json` de son dossier, ou son entrée du trousseau sous macOS	Non	Ce sont ceux de Claude Code. Escouade les lit pour savoir si un compte est connecté et si sa connexion est périmée. Elle n'en garde qu'une empreinte tronquée, ne copie pas le jeton et ne s'en sert pour aucune requête	`crates/escouade-engines/src/signin.rs:32-40`, `:53-58`, `:93-103`
Plugins installés, catalogue et liste de révocation en cache	`plugins/`, `store/`	Non	Empreinte de chaque version ; signature de la liste	`crates/escouade-plugins/src/lib.rs:255`, `crates/escouade-plugins/src/store.rs:414-453`
Mémoire partagée, spécification, décisions d'un projet	Dans le dépôt du projet (`.escouade/`)	**Oui** : ce sont des fichiers du projet, partagés par git	Ceux du dépôt	—
Identifiant d'installation des statistiques	Un fichier du dossier de données, créé à l'accord, supprimé au refus	Non	—	`src-tauri/src/telemetry.rs:355-380`

Escouade ne chiffre pas ses données au repos : elle s'en remet au chiffrement du disque du poste et aux droits du profil. C'est le choix habituel d'un outil de développement, à couvrir par la politique de chiffrement des postes de l'entreprise.

5. Les constats

Dix-huit constats, du plus grave au moins grave. Chaque fiche donne la gravité et sa raison, l'état de la preuve, le composant et l'effort de correction, puis la description, la preuve, un scénario réaliste et la recommandation. Un constat que l'éditeur connaît et laisse en l'état le dit dans sa fiche (SEC-03).

Id	Titre	Gravité	État
SEC-01	Plugins tiers : installables sans revue hors du store, et capables de faire lancer un programme	Moyenne	Démontré
SEC-02	« Projet privé » est une consigne, pas un cloisonnement	Moyenne	Démontré
SEC-03	Sceau « Vérifié par Escouade » : une seule clé, sans mot de passe, et une revue par un agent	Moyenne	Probable
SEC-04	Dépendances : des avis ouverts hors de l'application, sans délai de correction	Faible	Démontré
SEC-05	Politique de contenu : styles en ligne autorisés, directives manquantes	Faible	Démontré
SEC-06	Lancement décrit par un plugin : un fichier écrit à travers un lien	Faible	Démontré
SEC-07	Le jeton du serveur MCP passe en argument et reste en clair chez les harness	Faible	Probable
SEC-08	Flux d'un harness : une longueur annoncée suffit à arrêter l'application	Faible	Démontré
SEC-09	Réseau des plugins : trois angles morts	Faible	Démontré
SEC-10	Données au repos : pas de trousseau sous Linux, dossier lisible sous Unix	Faible	Probable
SEC-11	La sortie d'un plugin n'est pas plafonnée pendant un tour	Faible	Démontré
SEC-12	Fenêtre principale : des droits très larges si un script y entrait	Faible	Probable
SEC-13	Des variables d'environnement de test restent actives en version publiée	Faible	Probable
SEC-14	Mises à jour : le proxy est contourné s'il ne répond pas	Faible	Probable
SEC-15	Liste de révocation des plugins : pas de date de péremption	Faible	Théorique
SEC-16	Site de l'éditeur sans en-têtes de sécurité	Faible	Démontré
SEC-17	Serveurs locaux : un programme du poste peut les ralentir	Faible	Démontré
SEC-18	Observations diverses	Info	Observé

SEC-01 — Plugins tiers : installables sans revue hors du store, et capables de faire lancer un programme

Gravité	**Moyenne** — il faut que l'utilisateur installe le plugin et accepte ce que la fenêtre d'installation montre ; le programme lancé tourne alors hors du bac à sable, avec ses droits
État	Démontré pour la portée du réglage (essai B-15) et pour la liste de refus (test W-25) ; probable pour le reste (lu)
Composant	Installation des plugins ; harness décrits par un manifeste
Effort de correction	Moyen

Description. Trois faits, à lire ensemble.

1. **« N'installer que des versions vérifiées » ne couvre pas tout.** Le réglage est allumé pour une installation neuve (`crates/escouade-model/src/model.rs:769`). Une installation mise à jour depuis une version antérieure le garde éteint (`crates/escouade-model/src/model.rs:180-185`). Et il ne juge que les versions du store (`src-tauri/src/commands.rs:386-404`) : un plugin tiers s'installe toujours depuis un dossier, un dépôt git ou « en développement » (`:311`, `:443`, `:455`), sans que personne l'ait relu.
2. **Un plugin sans code peut décrire un harness** : un nom de programme, ses arguments, ses variables. Escouade cherche ce programme dans le `PATH` et le lance comme un agent, hors du bac à sable. Pour un tiers, le programme doit être un nom nu, jamais un chemin, et ne peut être ni un shell ni un interpréteur connu : une liste de 28 noms et de 6 familles (`crates/escouade-plugin-cli/src/manifest.rs:526-569`, `:1466-1487` ; `src-tauri/src/plugins/harness.rs:600-609`). C'est une liste de refus : un autre programme qui exécute ce que ses arguments disent passe.
3. **Ce que la fenêtre d'installation montre est alors le seul contrôle.** Elle montre chaque ligne de commande que le plugin déclare, les variables qu'il pose ou retire, les commandes d'installation et de mise à jour de son harness ; une mise à jour qui en change une redemande l'accord.

Preuve. Essai B-15 : le réglage allumé, le plugin d'essai, non signé, s'installe depuis un dossier, puis en développement. Test W-25 : les 31 noms de shells et d'interpréteurs essayés sont refusés, avec ou sans `.exe`, en toute casse ; 49 autres lanceurs passent, dont `py`, `pythonw`, `pwsh-preview`, `npx`, `uvx`, `dotnet`, `java` et `make`.

Scénario. Un collègue ou une page web recommande un plugin « harness pour l'outil X », à installer depuis son dépôt. Son manifeste déclare le programme `npx` et, comme arguments, un paquet à télécharger et à exécuter. L'utilisateur voit la ligne, ne la lit pas, accepte. Le paquet tourne avec ses droits au premier agent lancé sur ce harness.

Ce qui atténue. Rien ne s'installe sans le geste de l'utilisateur, et la fenêtre montre la ligne. La commande qui lit la version d'un harness décrit ne tourne ni à l'installation ni à chaque liste des harness : elle attend le démarrage d'un agent ou d'un terminal de ce harness (`src-tauri/src/plugins/harness.rs:1446`, `src-tauri/src/engine/registry.rs:413-430`). Un harness tiers n'entre dans un projet privé qu'activé par l'utilisateur pour ce projet (SEC-02). Les plugins de l'éditeur, livrés avec l'application, sont signés.

Recommandation. Pour un poste d'entreprise : allumer « N'installer que des versions vérifiées », et n'installer aucun plugin tiers, d'aucune provenance. Pour le produit : étendre le réglage aux dossiers et aux dépôts, ou dire sa portée dans son libellé ; l'allumer à la mise à jour d'une installation qui n'a aucun plugin tiers ; pour un harness décrit par un tiers, remplacer la liste de refus par un accord demandé au premier lancement, qui redise la ligne de commande.

SEC-02 — « Projet privé » est une consigne, pas un cloisonnement

Gravité	**Moyenne** — le nom de la fonction promet qu'un agent d'un projet ne lit pas un autre projet ; côté fichiers, elle ne le garantit pas
État	Démontré pour le contrôle des lancements (test W-04) ; probable pour le reste (lu)
Composant	`src-tauri/src/fence.rs`, `src-tauri/src/plugins/harness.rs`
Effort de correction	Élevé pour un vrai cloisonnement

Description. Marquer un projet « privé » cache ce projet aux agents des autres projets. Côté serveur MCP, c'est tenu par le cœur. Côté fichiers, la « clôture » est une liste de chemins passée au harness comme règles de refus. Trois limites :

1. Sous Windows natif, Claude Code n'a pas de bac à sable : la règle s'applique aux outils de lecture et d'écriture, et aux commandes dont le harness sait lire les chemins. Une commande qui construit son chemin autrement lit le dossier.
2. Pour un harness servi par un plugin tiers, le cœur vérifie que la clôture « est dans le lancement » en cherchant chaque chemin comme une simple sous-chaîne des arguments, des variables ou des fichiers. Un chemin posé dans une variable que personne ne lit satisfait le contrôle.
3. Sans aucun projet privé, il n'y a aucune clôture : un agent lit tout ce que l'utilisateur lit.

Preuve. `src-tauri/src/fence.rs:281-306` : les règles `permissions.deny`, et `sandbox.filesystem` seulement hors de Windows natif ; `:13` : sans projet privé, rien n'est refusé. `src-tauri/src/plugins/harness.rs:1114-1124` :

```
let said = |p: &str| { … launch.args.iter().any(|a| plain(a).contains(p))
    || launch.env.iter().any(|v| plain(&v.value).contains(p))
    || launch.files.iter().any(|f| plain(&f.text).contains(p)) };
let held = (first_hand && launch.fence_by_client)
    || (launch.fence_applied && fence.iter().all(|p| said(p)));
```

Test W-04 : un lancement qui pose le chemin protégé dans une variable sans effet, ou dans un argument qui l'**autorise**, passe le contrôle.

Scénario. L'utilisateur tient sur le même poste deux projets qui ne doivent pas se voir, et marque chacun « privé ». Un agent du premier, détourné par un contenu piégé, lance une commande qui lit un fichier du second par un chemin que les règles ne reconnaissent pas.

Ce qui atténue. L'interface dit la limite : sous Windows, la description du réglage écrit que « la clôture d'un agent est une consigne donnée à son harness, pas un cloisonnement du système : une commande peut la contourner », et conseille un compte Windows par ensemble de projets à séparer (`src/lib/i18n/fr/settings.ts:491`). Sous macOS, Linux et WSL, le bac à sable de Claude Code applique la clôture à toutes les commandes. Le harness d'un plugin tiers n'est lancé dans un projet privé qu'une fois activé par l'utilisateur pour ce projet (`src-tauri/src/harness_scope.rs:250-253`). Le lancement d'une vue de terminal est refusé par le cœur s'il n'applique pas la clôture (`src-tauri/src/plugins/harness.rs:1111-1131`) ; la vue Copilot CLI, qui ne sait pas la tenir, se refuse elle-même à un agent clôturé (`plugins/copilot/src/terminal.rs:178-186`, test W-29).

Recommandation. Pour un poste d'entreprise : un poste, ou un compte du système, par ensemble de projets à séparer. Pour le produit : pour un plugin tiers, remplacer la recherche d'une sous-chaîne par une preuve vérifiable, ou refuser le lancement dans un projet privé ; à terme, un bac à sable du système sous Windows.

SEC-03 — Sceau « Vérifié par Escouade » : une seule clé, sans mot de passe, et une revue par un agent

Gravité	**Moyenne** — la clé sans mot de passe signe la catégorie de plugins à laquelle le cœur accorde le plus ; l'exploitation demande la clé, un moyen de faire servir la version forgée, et un clic de l'utilisateur (voir « Pourquoi cette gravité »)
État	Probable — la pratique est lue dans la documentation et l'outillage du dépôt, et précisée par l'éditeur ; la portée de la clé est vérifiée dans le code ; le dossier des clés n'a pas été lu, par consigne
Composant	Revue et signature des plugins du store
Effort de correction	Moyen
Position de l'éditeur	Accepté — l'éditeur connaît ce point, garde ce fonctionnement et le dit

Description. Une version de plugin « vérifiée » porte une signature minisign que l'application contrôle avec deux clés publiques embarquées : la clé de tous les jours et une clé de secours. Le modèle de menace du store décrit la clé secrète comme « chiffrée par mot de passe, hors ligne en dehors des signatures ». En pratique, la clé de tous les jours est gardée sans mot de passe sur le poste de l'éditeur, pour qu'un agent automatisé signe seul les plugins de première main. La revue qui précède cette signature est faite, elle aussi, par un agent d'IA.

L'éditeur a précisé ce point pendant l'audit : la clé sans mot de passe n'a signé que ses propres plugins, liés au cœur d'Escouade ; pour les plugins soumis par d'autres auteurs, la signature demandera un mot de passe. Cette fiche sépare ce qui se vérifie de ce qui est promis.

Ce qui est fait, et vérifié.

- **La clé sans mot de passe n'a signé que des plugins de première main** (`dev.escouade.*`) : ceux de l'éditeur, liés au cœur et livrés avec l'application. La procédure du dépôt le décrit, et le catalogue du store, lu le 12 novembre 2026, le confirme : 28 plugins, tous `dev.escouade.*` ; 89 versions vérifiées, aucune hors de cet espace (essai N-02).
- **La revue d'un plugin tiers ne signe pas.** La commande de revue l'interdit à l'agent : « la signature est le geste de l'utilisateur, sur sa décision ».
- **Il n'y a qu'une clé, la même pour tous.** Le sceau d'un plugin tiers passerait aujourd'hui par la clé qui signe la première main. Dans l'application, une signature d'une des deux clés embarquées vaut pour n'importe quel identifiant de plugin : rien ne réserve la clé de tous les jours à `dev.escouade.*`. Dans l'outillage, la commande `sign` ne connaît qu'un fichier de clé, quel que soit le plugin, et la publication de la première main appelle cette même commande. Cette clé signe aussi la liste de révocation.

Ce qui est annoncé (engagement de l'éditeur, non vérifié). La signature des plugins soumis par des tiers se fera avec une clé protégée par un mot de passe. À la date de l'audit, aucun plugin tiers n'est signé : rien ne contredit cet engagement, et rien ne le démontre. Ce n'est donc pas un contrôle en place. Ce qui permettrait de le vérifier plus tard :

1. dans le code de l'application, une clé publique distincte pour les tiers, et la clé de la première main limitée aux identifiants `dev.escouade.*`, avec un test qui montre qu'elle ne scelle rien d'autre ;
2. sur chaque sceau, l'identifiant de la clé qui a signé (il figure dans le fichier de signature) : quiconque peut alors contrôler quelle clé a scellé quel plugin ;
3. une procédure écrite, et tenue, qui dit où vit chaque clé, qui la détient et comment elle est protégée.

Preuve.

```
crates/escouade-plugins/src/store_keys.rs:16-19    deux clés publiques embarquées : tous les jours, secours
crates/escouade-plugins/src/store_keys.rs:115-134  verified_with : toute clé, pour tout identifiant de plugin
store/scripts/plugin-review/admin.ts:52-53   un seul fichier de clé (store.key), quel que soit le plugin
store/scripts/plugin-review/admin.ts:189-229 signVersion : la même clé pour tout identifiant
store/scripts/plugin-review/admin.ts:326     la liste de révocation, signée par cette même clé
store/scripts/plugins-publish/publish.ts:238 la première main appelle la même commande sign
.claude/commands/review-plugin.md:17         la revue d'un tiers : « Ne signe pas », le geste reste à
l'utilisateur
docs/STORE.md:465   « Sa clé secrète est créée sur le poste […], chiffrée par un mot de passe »
docs/STORE.md:631   « Voler la clé de signature | chiffrée par mot de passe, hors ligne… »
docs/STORE.md:760   « puis sign (la clé du store, sans mot de passe…) »
docs/STORE.md:395   « Côté poste : un agent dans ce dépôt » (la revue)
```

Scénario. Le poste de l'éditeur est compromis, ou un agent qui y tourne est détourné par un contenu qu'il lit. La clé étant lisible sans mot de passe, l'attaquant signe ce qu'il veut : une version piégée d'un plugin de première main, ou un plugin sous n'importe quel autre identifiant, puisque l'application ne distingue pas. Il lui reste à faire servir cette version (un store compromis, ou une interception du réseau). Elle porte le sceau, passe le réglage « vérifiées seulement », et s'installe sur un clic de mise à jour. Un plugin de première main forgé est le pire cas : c'est la catégorie à laquelle le cœur accorde le plus. Sa parole sur la clôture des projets privés est crue (`src-tauri/src/plugins/harness.rs:1123`), et ses variables de lancement ne sont tenues qu'à ce que son propre manifeste déclare, sans les familles refusées à un tiers (`:1156-1171`).

Pourquoi cette gravité. Trois raisons de ne pas la mettre plus bas :

1. La précision de l'éditeur borne l'usage, pas la capacité. Que la clé n'ait servi qu'à la première main est exact et vérifié ; qu'elle ne puisse servir qu'à cela ne l'est pas, tant que le code accepte sa signature pour tout plugin et pour la liste de révocation.
2. Ce qu'elle signe est la catégorie la plus privilégiée, et la seule que ce rapport recommande d'installer. Ces plugins décrivent le lancement de chaque harness, son interface et les lectures de son compte (section 6.9) : ce que le sceau protège pèse d'autant.
3. La protection annoncée pour les tiers est un engagement, pas encore un contrôle.

Et une raison de ne pas la monter : l'exploitation demande toujours trois conditions réunies, à savoir la clé, un moyen de faire servir la version forgée, et un clic de l'utilisateur. La gravité passerait à faible le jour où la clé de la première main serait limitée par le code à `dev.escouade.*` , protégée et hors de portée des agents, et où la liste de révocation serait signée par une clé protégée.

Ce qui atténue. Aucune mise à jour de plugin ne s'installe sans un clic. Une liste de révocation signée retire un sceau ou une clé (`crates/escouade-plugins/src/store.rs:308-335`), et la clé de secours, tenue hors ligne, peut révoquer la clé de tous les jours. Les plugins de première main livrés avec l'application viennent de l'installateur signé : ils s'installent sans réseau, leur sceau contrôlé sur le poste (`src-tauri/tauri.conf.json:42` , `crates/escouade-plugins/src/seed.rs:149-165`).

Ce que le sceau veut dire. Le sceau « Vérifié par Escouade » d'un plugin de première main dit « publié par l'éditeur d'Escouade », après une revue faite par un agent : il ne dit pas « relu par un humain ». C'est le même niveau de confiance que celui accordé à l'application elle-même, dont ces plugins sont des morceaux.

Recommandation. Rendre l'engagement vérifiable : une clé distincte et protégée pour les tiers, la clé de la première main limitée par le code à `dev.escouade.*` , la liste de révocation signée par une clé protégée. Tenir toute clé hors de portée des agents qui lisent du contenu tiers. Faire relire par un humain toute version avant signature, ou dire sur la page du plugin que la revue est automatisée. Aligner `docs/STORE.md` (§4.5 et §7) sur la pratique. Pour un poste d'entreprise : s'en tenir aux plugins de première main livrés avec l'application.

SEC-04 — Dépendances : des avis ouverts hors de l'application, sans délai de correction

Gravité	**Faible** — aucun avis n'est atteignable dans l'application ; ce qui reste ouvert touche le serveur du store et la construction du site, hors du poste, et quatre avis écartés dont la raison écrite n'est plus exacte
État	Démontré — outils lancés pendant l'audit, sorties en annexes A et B
Composant	`deny.toml`, `.github/workflows/advisories.yml`, dépendances du store et du site
Effort de correction	**Faible**

Description. Les avis de sécurité des dépendances sont contrôlés en CI : `cargo deny check advisories` (application, plugins) et `npm audit --omit=dev` (application, store, site), à chaque pull request, à chaque poussée sur `main` et sur les branches de publication, et chaque nuit (`.github/workflows/advisories.yml:13`, `:47`, `:76`). Dependabot propose les mises à jour, groupées, chaque mois (`.github/dependabot.yml:7-87`). Pour l'application, le contrôle est bloquant, et aucun avis n'y est atteignable. Trois points restent.

1. **Quatre avis Rust sont écartés pour une raison qui n'est plus exacte.** Ils visent `zip` 4.6.1 (la fonction `extract`). La raison écrite, « l'application n'appelle jamais `extract` : `docsextract.rs` lit les entrées en mémoire » (`deny.toml:17-20`), ne décrit pas ce qui porte cette version : le code de l'application emploie `zip` 9.5, et `zip` 4.6.1 n'est tiré que par le module de mise à jour (`tauri-plugin-updater` 2.13.1). Celui-ci appelle `extract` sous Windows quand la mise à jour reçue est une archive zip, après avoir vérifié la signature de l'éditeur ; les mises à jour publiées pour Windows sont des installateurs, pas des archives. Le risque est négligeable ; la raison est à récrire.
2. **Les avis npm du store (14) et du site (10) ne font pas échouer la CI** (`.github/workflows/advisories.yml:59-65`), et aucun délai cible de correction n'est écrit. Store : 6 faibles et 8 moyens, sur `hono`, `@hono/node-server`, `glob` et `minimatch` ; c'est un serveur qui reçoit des archives et des images de tiers. Site : 4 faibles et 6 moyens, sur des bibliothèques qui ne tournent qu'à sa construction. Aucun avis élevé ni critique.
3. **Trois avis visent une bibliothèque qui n'est pas compilée** (RUSTSEC-2026-0523, -0524, -0538, sur `quinn-proto` 0.11.18 et `quinn-udp` 0.5.15 : `src-tauri/Cargo.lock:4856`, `:4878`). Ces bibliothèques du protocole QUIC sont dans les fichiers de verrouillage comme dépendance optionnelle de `reqwest`, pour HTTP/3, que l'application n'active pas : `cargo tree -i quinn-proto` ne les trouve dans le graphe d'aucune cible, toutes fonctions de compilation activées. `cargo audit`, qui lit le fichier de verrouillage, les cite ; `cargo deny`, qui juge ce qui est construit, non. Sans effet : à connaître pour lire les sorties.

Preuve. Les outils, lancés sur le commit audité le 14 novembre 2026 (annexes A et B) :

Outil	Résultat
`cargo audit`, application	7 vulnérabilités : les quatre écartées avec leur raison, les trois sur la bibliothèque qui n'est pas compilée
`cargo audit`, plugins	3, ces trois-là
`cargo deny check`	avis, interdictions, licences, sources : ok, pour l'application et pour les plugins
`npm audit --omit=dev`, application	0
`npm audit --omit=dev`, store	14 (6 faibles, 8 moyennes), non bloquant en CI
`npm audit --omit=dev`, site	10 (4 faibles, 6 moyennes), non bloquant en CI

Scénario. Une faille exploitable est publiée dans une bibliothèque du serveur du store. La CI la signale chaque nuit sans échouer ; sans délai écrit, la correction attend. Le store ne détient aucune clé de signature et ne peut pas forger un sceau (section 6.8), mais c'est lui qui sert le catalogue, les versions non vérifiées et la liste de

révocation (SEC-15).

Recommandation. Récrire la raison des quatre avis écartés. Écrire un délai cible de correction (7 jours pour un avis élevé), et rendre bloquants en CI les avis élevés du store et du site.

SEC-05 — Politique de contenu : styles en ligne autorisés, directives manquantes

Gravité	**Faible** — défense en profondeur : il faut d'abord une faille de l'assainissement, que les essais n'ont pas trouvée
État	**Démontré** — essai B-07
Composant	`src-tauri/tauri.conf.json:32`
Effort de correction	Moyen (les styles en ligne viennent de l'interface elle-même)

Description. La politique de contenu de la fenêtre bloque les scripts en ligne, `eval`, les scripts et les connexions vers l'extérieur, les images et cadres distants : quinze tentatives bloquées à l'essai. Elle autorise en revanche les styles en ligne (`style-src 'unsafe-inline'`), et ne fixe ni `form-action`, ni `base-uri`, ni `frame-ancestors`.

Preuve.

```
"csp": "default-src 'self' ipc: http://ipc.localhost; img-src 'self' asset: http://asset.localhost data:
blob:;
        style-src 'self' 'unsafe-inline'; font-src 'self' data:; script-src 'self' 'wasm-unsafe-eval';
        connect-src 'self' ipc: http://ipc.localhost"
```

Essai B-07 : les quinze tentatives de script, de connexion et de chargement sont bloquées ; une balise `<style>` injectée dans la page s'applique.

Scénario. Si un contenu tiers parvenait à injecter du HTML, il ne pourrait pas exécuter de script, mais pourrait redessiner l'interface : recouvrir un bouton « Refuser » par un faux « Autoriser », par exemple.

Recommandation. Ajouter `form-action 'none'`, `base-uri 'none'`, `object-src 'none'`, `frame-ancestors 'none'`. Viser la suppression de `'unsafe-inline'` pour les styles (styles par classes, ou empreintes).

SEC-06 — Lancement décrit par un plugin : un fichier écrit à travers un lien

Gravité	**Faible** — il faut la coopération d'un plugin de harness et d'un lien déjà présent dans le dossier de l'agent
État	Démontré — test W-02
Composant	`src-tauri/src/plugins/harness.rs:1246-1305`
Effort de correction	**Faible**

Description. Un plugin qui sert un harness peut faire écrire des fichiers au cœur avant le lancement, dans le dossier propre de l'agent. Le chemin est jugé tel qu'il est écrit (absolu, sous le dossier, sans `..`), puis écrit en suivant les liens. Si le dossier contient un lien ou une jonction vers l'extérieur, le fichier atterrit dehors. Second point : à côté du fichier privé d'une interface, un plugin peut écrire tout nom qui commence par le même radical, dont un nom de script (`settings.cmd` à côté de `settings.json`).

Preuve. `check_file` ne résout pas les liens ; `write_files` appelle `std::fs::write` (`src-tauri/src/plugins/harness.rs:1246-1260`, `:1295-1304`). Les capacités `fs/*` des plugins, elles, résolvent les liens avant de juger (`crates/escouade-plugins/src/files.rs:607-619`) : l'écart est entre les deux chemins de code. Test W-02 : une jonction `cache` dans le dossier de l'agent, un fichier `cache/…` dans le lancement : accepté, écrit hors du dossier. Test W-03 : à côté du fichier privé d'une interface, tout nom qui commence par son radical est accepté (`src-tauri/src/plugins/harness.rs:1264-1292`).

Scénario. Un agent détourné crée une jonction dans son dossier de configuration. Au lancement suivant, le plugin de son harness y écrit un fichier, qui remplace un fichier ailleurs sur le poste.

Recommandation. Résoudre les liens avant de juger, comme `fs/*` le fait. Restreindre `beside` aux extensions attendues.

SEC-07 — Le jeton du serveur MCP passe en argument et reste en clair chez les harness

Gravité	**Faible** — sur un poste à un seul utilisateur, seuls les programmes de sa propre session lisent le jeton ; la gravité serait moyenne sur une machine partagée sous Linux ou macOS
État	Probable — lu dans le code ; un test du projet affirme la ligne de commande
Composant	Déclaration du serveur MCP dans les harness
Effort de correction	Moyen (dépend de ce que chaque harness accepte)

Description. Pour que Claude Code, dans un terminal, pilote Escouade, l'application déclare son serveur dans la configuration du harness. Elle le fait par la commande `claude mcp add … --header "Authorization: Bearer <jeton>"` : le jeton est un argument de processus, lisible par les autres programmes de l'utilisateur et, sous Unix, par tous les comptes de la machine pendant l'exécution. Il est ensuite écrit en clair dans le fichier de configuration du harness, alors que l'application le garde elle-même au trousseau.

Preuve. `src-tauri/src/engine/claude/mcp.rs:72-82` (les arguments), `:20-30` (l'entrée écrite). Le cœur refuse pourtant qu'un plugin mette ce jeton dans les arguments d'un harness, pour cette raison même (`src-tauri/src/plugins/harness.rs:1103-1109`).

Scénario. Sur un serveur de développement partagé, un autre compte lit la liste des processus au moment de la déclaration, obtient le jeton, se connecte au port local et crée un agent avec ses propres consignes dans un projet de la victime.

Ce qui atténue. Le serveur est éteint par défaut. Éteindre l'option change le jeton à l'instant (`src-tauri/src/mcp/mod.rs:267-280`). Les jetons des agents, eux, ne passent ni par un argument ni par un fichier durable : celui d'un agent est retiré des fichiers de son interface quand elle se ferme (`src-tauri/src/tui.rs:782-810`). Copilot CLI reçoit le jeton par un fichier privé, pas dans une variable d'environnement dont ses commandes hériteraient (`plugins/copilot/src/terminal.rs:230-236`).

Recommandation. Écrire l'entrée directement dans le fichier de configuration du harness plutôt que par la ligne de commande. Quand le harness le permet, référencer une variable d'environnement plutôt que le jeton. Documenter que tout programme du poste qui lit ce fichier pilote Escouade.

SEC-08 — Flux d'un harness : une longueur annoncée suffit à arrêter l'application

Gravité	**Faible** — déni de service par un harness, donc par un programme que l'utilisateur a choisi de lancer
État	Démontré — tests W-14 et W-15
Composant	`src-tauri/src/plugins/harness.rs:1307-1331` (`read_frame`)
Effort de correction	**Faible**

Description. Certains harness parlent au cœur par des messages précédés d'un en-tête `Content-Length`. Le lecteur alloue un tampon de la taille annoncée avant d'avoir lu un seul octet du message. Une taille plus grande que la mémoire disponible fait échouer l'allocation, ce qui arrête le processus : ce n'est ni une erreur ni une panique rattrapable.

Preuve.

```
if let Some(n) = len {
    let mut body = vec![0; n];
    r.read_exact(&mut body).await?;
```

Tests W-14 et W-15 : un en-tête de 29 octets fait allouer 256 Mio. Avec une longueur de 2^62, le processus d'essai s'arrête avec le code `0xC0000409` et le message « memory allocation of 4611686018427387904 bytes failed ».

La même faiblesse a un second endroit, lu dans le code : quand un plugin lit un compte par un process (`account/read`), chaque ligne que le process écrit est lue sans plafond de longueur (`src-tauri/src/plugins/harness.rs:912-923`), pendant 60 secondes au plus (`:961-963`). Cette lecture sert aussi au quota de Claude Code, toutes les cinq minutes. Non essayé : le process est le harness du compte, pas un programme qu'un tiers choisit.

Scénario. Un harness décrit par un plugin tiers, ou un harness défaillant, écrit une longueur absurde. L'application s'arrête, avec tous les agents.

Recommandation. Plafonner la longueur d'un message (quelques dizaines de Mio) et lire par morceaux.

SEC-09 — Réseau des plugins : trois angles morts

Gravité	**Faible** — le contrôle principal tient ; ce sont ses bords
État	Démontré — tests W-11 et W-12 ; un test du projet pour le proxy
Composant	`crates/escouade-plugins/src/wasm.rs` (`net/fetch`, `net/stream`, sockets)
Effort de correction	Faible à moyen

Description. Un plugin ne joint que les hôtes de sa permission `network`, en https, sans adresse écrite, sans redirection suivie, et jamais une adresse de la machine ou du réseau local : c'est jugé sur les adresses que le nom résout, à la connexion même, ce qui tient contre le DNS rebinding (essais W-09 et W-10). Trois angles morts :

1. **À travers un proxy, seul le nom est jugé.** Quand un proxy est configuré, c'est lui qui résout le nom ; l'application ne voit plus l'adresse. Un plugin dont l'hôte déclaré pointe, vu du proxy, vers une adresse interne, l'atteint si le proxy l'autorise. C'est précisément la configuration d'un poste d'entreprise.
2. **Un domaine de premier niveau entier est un hôte acceptable.** `permissions.network: ["com"]` couvre tous les `.com`, et l'écran d'installation affiche « com ».
3. Une adresse IPv4 portée dans une adresse 6to4 ou Teredo n'est pas reconnue comme locale (sans effet pratique connu).

Preuve. `crates/escouade-plugins/src/wasm.rs:25` (« through a proxy only the name is judged, and the journal says so »), `:1049-1064` (`covers`), `:805-830` (`forbidden`). Tests W-11 et W-12 : une adresse portée en 6to4 ou Teredo n'est pas jugée, et un domaine de premier niveau entier est accepté comme hôte.

Scénario. Un plugin tiers déclare son propre domaine, que son auteur fait pointer vers `10.0.0.5`. Sur un poste derrière un proxy interne, le plugin interroge un service de l'intranet et renvoie la réponse à son auteur par le même canal.

Recommandation. Refuser à la lecture du manifeste un hôte sans point ou un suffixe public. Derrière un proxy, dire à l'installation que les adresses ne sont plus contrôlées, ou résoudre le nom localement en plus pour refuser les adresses privées.

SEC-10 — Données au repos : pas de trousseau sous Linux, dossier lisible sous Unix

Gravité	**Faible** — concerne une machine partagée entre plusieurs comptes, sous Linux ou macOS
État	Probable — lu dans le code ; Linux et macOS n'ont pas été essayés
Composant	`crates/escouade-forge/src/integrations/secrets.rs`, `src-tauri/src/paths.rs`
Effort de correction	Moyen

Description. Sous Windows et macOS, les jetons vont au trousseau du système. Sous Linux, l'application n'utilise aucun trousseau : les jetons restent dans un fichier du dossier de données, lisible par son propriétaire seul. Par ailleurs, seuls les fichiers qui tiennent un secret sont créés en mode privé. Le dossier de données et les conversations suivent le masque par défaut du système : sur une machine Unix partagée, un autre compte peut lire les conversations, donc des extraits du code des projets. Enfin, l'adresse du proxy, mot de passe compris si l'utilisateur l'y a mis, est écrite en clair dans les réglages et passée en variable aux agents.

Preuve. `crates/escouade-forge/src/integrations/secrets.rs:98-121` (`NoKeychain` : « the secrets stay in the file »), `crates/escouade-model/src/paths.rs:691-713` (le mode 0600 n'est posé que par `write_private`), `crates/escouade-model/src/model.rs:35` (le proxy avec ses identifiants).

Scénario. Sur une machine de développement Linux partagée, un autre utilisateur parcourt le dossier de données de la victime et lit les conversations de ses agents.

Recommandation. Créer le dossier de données en mode 0700. Sous Linux, utiliser le service de secrets du bureau quand il existe. Garder le mot de passe du proxy au trousseau.

SEC-11 — La sortie d'un plugin n'est pas plafonnée pendant un tour

Gravité	**Faible** — pic de mémoire transitoire, borné par la durée d'un tour et par le nombre de relances
État	Démontré — essai B-12
Composant	Hôte WebAssembly, `crates/escouade-plugins/src/wasm.rs`
Effort de correction	**Faible**

Description. La mémoire d'un plugin est bornée (64 Mio, 256 au plus) et un tour de calcul est coupé après 2 secondes. Ce que le plugin **écrit** pendant ce tour n'a pas de plafond : tout est gardé en mémoire par l'application jusqu'à la fin du tour.

Preuve. `crates/escouade-plugins/src/wasm.rs:431-437` : chaque écriture du module est ajoutée à un tampon en mémoire, sans limite, jusqu'à la fin du tour. Essai B-12 : un plugin qui écrit sans fin fait monter la mémoire privée de l'application de 19 Mio à un pic de 312 Mio, avant d'être arrêté ; le pic dépend de la charge du poste. Après quatre arrêts en cinq minutes, l'hôte ne le relance plus, et la mémoire redescend à 44 Mio.

Scénario. Un plugin malveillant ou défaillant provoque quatre pics successifs de plusieurs centaines de Mio. Sur un poste déjà chargé, le système peut manquer de mémoire. L'application, elle, a survécu à l'essai.

Recommandation. Plafonner les octets écrits par tour (quelques Mio) et arrêter l'instance au-delà.

SEC-12 — Fenêtre principale : des droits très larges si un script y entrait

Gravité	**Faible** — défense en profondeur ; aucun moyen d'injecter un script n'a été trouvé
État	Probable — lu dans la configuration
Composant	`src-tauri/capabilities/default.json` , `src-tauri/src/lib.rs:650-999`
Effort de correction	Moyen

Description. La fenêtre principale peut appeler 348 commandes du cœur, sans découpage : terminaux, fichiers, git, plugins, réglages. Elle peut aussi demander au système d'ouvrir n'importe quel chemin du disque avec son programme par défaut. C'est normal pour l'interface d'un outil de développement, mais cela veut dire qu'une seule exécution de script dans la fenêtre vaudrait un contrôle complet du poste. Toute la sécurité de cette frontière repose sur l'assainissement du contenu et la politique de contenu.

Preuve.

```
{ "identifier": "opener:allow-open-path", "allow": [{ "path": "**" }, { "path": "$HOME/.escouade*" }] }
```

`src-tauri/capabilities/default.json:21-22` ; la liste des commandes : `src-tauri/src/lib.rs:650-999`.

Scénario. Une faille de la bibliothèque d'assainissement, ou un futur composant qui afficherait du HTML tiers sans passer par elle, laisse un message d'agent exécuter un script dans la fenêtre. Ce script ouvre un terminal par une commande du cœur, ou fait ouvrir par le système un exécutable qu'un agent a déposé dans le projet : il agit avec les droits de l'utilisateur.

Ce qui tient. La page de l'application en cours de développement s'affiche dans une webview à part, à laquelle aucune commande ne répond (`src-tauri/src/windowthread.rs:224-231` , et la capacité ne nomme que la fenêtre principale). Les outils de développement de la fenêtre sont fermés dans une version publiée (`src-tauri/src/lib.rs:412`).

Recommandation. Restreindre `opener:allow-open-path` aux dossiers des projets et au dossier de données. Activer le mode d'isolation de Tauri, ou découper les commandes par capacité, pour qu'un script injecté n'ait pas tout d'un coup.

SEC-13 — Des variables d'environnement de test restent actives en version publiée

Gravité	**Faible** — il faut pouvoir régler l'environnement de l'application, donc déjà agir dans la session de l'utilisateur
État	Probable — lu dans le code
Composant	`crates/escouade-forge/src/forge/azure.rs:22-30`, `src-tauri/src/vectors/wiring.rs:26-31`
Effort de correction	**Faible**

Description. La plupart des variables qui détournent l'application vers un faux service de test ne sont lues que par un binaire de développement (store, mises à jour, clé de test du store). Deux ne le sont pas : `ESCOUADE_AZURE_DEVOPS_API` remplace l'adresse de l'API d'Azure DevOps, et l'application y enverrait le jeton du compte ; `ESCOUADE_VECTORS_MODEL_URL` remplace l'adresse de téléchargement d'un modèle (sans effet réel : le modèle est vérifié par son empreinte).

Preuve. `api_override` lit la variable sans condition de construction, contrairement à `store::debug_env` (`crates/escouade-plugins/src/store.rs:55`) ou à `updates::configured_endpoints` (`src-tauri/src/updates.rs:925-931`).

Scénario. Un script de démarrage de session, posé par un autre programme, définit la variable. Le jeton Azure DevOps de l'utilisateur part vers un serveur tiers à la prochaine lecture des pull requests.

Recommandation. Placer ces deux lectures derrière la même condition de construction que les autres.

SEC-14 — Mises à jour : le proxy est contourné s'il ne répond pas

Gravité	**Faible** — un écart de politique plus qu'une faille : le contenu reste signé
État	Probable — lu dans le code
Composant	`src-tauri/src/updates.rs:858-883`, `crates/escouade-updates/src/net.rs:9-12`
Effort de correction	**Faible**

Description. Si le proxy des réglages est injoignable (connexion refusée, délai, nom inconnu), le contrôle de mise à jour et le téléchargement tentent une fois une connexion directe. Un proxy qui **répond** par un refus n'est jamais contourné : le code le dit et s'y tient. Dans une entreprise où toute sortie doit passer par le proxy, la tentative directe est un écart, même si le pare-feu la bloquera.

Preuve. `Failure::proxy_unreachable`, puis `self.once(Route::Direct)` (`src-tauri/src/updates.rs:867-868`).

Scénario. Le proxy de l'entreprise est en panne ou injoignable depuis un réseau extérieur. L'application tente de joindre directement le serveur des mises à jour. Sur un réseau qui laisse sortir, la requête part sans passer par le filtrage ni la journalisation du proxy. Ce qui est téléchargé reste vérifié par sa signature.

Ce qui atténue. Ce qui est téléchargé reste vérifié par sa signature. Cette route directe vérifie toujours les certificats, même quand « Ignorer la vérification » est cochée (`crates/escouade-updates/src/net.rs:102-113`).

Recommandation. Un réglage « ne jamais sortir sans le proxy », allumé d'office quand un proxy est configuré.

SEC-15 — Liste de révocation des plugins : pas de date de péremption

Gravité	**Faible** — il faut contrôler le store ou le réseau, et le résultat est seulement de retarder un retrait
État	Théorique — lu dans le code
Composant	`crates/escouade-plugins/src/store.rs:308-335`
Effort de correction	**Faible**

Description. La liste qui retire un plugin ou une clé est signée et numérotée : une liste plus ancienne est refusée, une liste non signée aussi (essais W-13, W-20). Mais elle n'a pas de date limite. Qui peut empêcher l'application de recevoir la nouvelle liste (le store compromis, ou un attaquant sur le réseau) maintient l'ancienne en vigueur sans limite de temps, et l'application ne le signale pas.

Preuve. `crates/escouade-plugins/src/store.rs:308-335` (`check_list`) : la signature, le signataire et le numéro de série sont contrôlés ; aucun champ ne dit jusqu'à quand la liste vaut. `:643-703` : une liste qui n'arrive pas laisse la précédente en place, avec un message d'erreur dans la page du store seulement.

Scénario. Un plugin vérifié se révèle malveillant. L'éditeur publie une liste qui le retire. Un attaquant qui tient le réseau du poste, ou le serveur du store, sert l'ancienne liste (ou une erreur) : le plugin reste actif et son sceau reste affiché, aussi longtemps que l'attaquant tient sa position.

Ce qui atténue. Une copie de la liste est servie par un second hôte, lu quand le store ne répond pas.

Recommandation. Dater la liste, la republier régulièrement même inchangée, et avertir quand la dernière liste reçue a plus de quelques jours.

SEC-16 — Site de l'éditeur sans en-têtes de sécurité

Gravité	**Faible** — le site ne sert que des pages statiques, mais c'est de là que l'on télécharge l'application
État	Démontré — requêtes en lecture seule, annexe D (N-01 à N-06)
Composant	`website/nginx.conf`, hébergement du site
Effort de correction	**Faible**

Description. Le store de plugins répond avec un jeu complet d'en-têtes : politique de contenu à nonce, HSTS, `X-Content-Type-Options`, `X-Frame-Options`, politiques d'origine croisée, aucun en-tête CORS pour une origine étrangère, TLS 1.0 et 1.1 refusés. Le site de l'éditeur, lui, n'envoie **aucun** de ces en-têtes et annonce la version de son serveur. Sans HSTS, une première visite en `http` peut être détournée avant la redirection. Le store annonce aussi son cadriciel (`X-Powered-By`).

Preuve. Réponse du site, lue le 14 novembre 2026 : `Content-Type`, `Etag`, `Last-Modified`, `Server: nginx/<version>`, `Vary`, rien d'autre. `website/nginx.conf:12-57` : le bloc du site ne contient aucun `add_header` de sécurité (les trois qu'il a règlent le cache).

Scénario. Sur un réseau hostile, un utilisateur qui tape l'adresse du site sans `https://` est redirigé vers une copie qui propose un installateur piégé. La signature du système d'exploitation est alors le dernier rempart.

Recommandation. Ajouter HSTS, une politique de contenu, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, et masquer la version du serveur. Publier les empreintes des installateurs.

SEC-17 — Serveurs locaux : un programme du poste peut les ralentir

Gravité	**Faible** — déni de service local, par un programme de la même session
État	Démontré — essai B-03
Composant	`crates/escouade-mcp/src/http.rs:55-62`, `src-tauri/src/tui.rs:2802-2829`
Effort de correction	**Faible**

Description. Le serveur MCP accepte 64 connexions à la fois et ferme celles qui n'envoient pas leurs en-têtes en 10 secondes. Un programme sans jeton qui ouvre des connexions muettes retarde donc les appelants légitimes : 8,5 secondes d'attente mesurées derrière 80 connexions, et indéfiniment s'il les rouvre en boucle. Le serveur de hooks n'a pas de plafond de connexions, et compare ses jetons par une recherche ordinaire plutôt qu'en temps constant (jeton de 244 bits : sans conséquence pratique).

Preuve. `crates/escouade-mcp/src/http.rs:55-62` (64 connexions, 10 secondes pour les en-têtes), `src-tauri/src/tui.rs:2802-2829` (aucun plafond), `:128-130` (recherche du jeton dans une table). Essai B-03 : 80 connexions muettes, la requête légitime suivante attend 8 493 ms.

Scénario. Un programme quelconque de la session, sans aucun jeton, ouvre des connexions muettes en boucle. Les agents et le terminal de l'utilisateur ne joignent plus le serveur MCP : les outils d'Escouade échouent ou attendent, sans autre effet.

Recommandation. Réduire le délai des en-têtes à 2 secondes pour une connexion locale ; plafonner les connexions du serveur de hooks.

SEC-18 — Observations diverses

Gravité	Info — sans impact de sécurité direct ; à connaître
État	Observé pendant les essais
Composant	Divers
Effort de correction	Faible, au cas par cas

- **Noms réservés de Windows.** Par `fs/write`, un plugin peut créer dans le dossier de son agent un fichier nommé `NUL`, `CON` ou `COM1`. Le fichier reste dans le dossier permis, mais les outils courants de Windows ne savent ni l'ouvrir ni le supprimer (test W-08).
- **Image d'un chemin local.** Un message d'agent peut faire afficher une image située hors du projet, si l'utilisateur peut la lire. Seules les images sont lues, et rien ne sort du poste (essai B-08).
- **Image distante au clic.** Le téléchargement se fait en https, sans cookie ni référent, mais sans refuser les adresses du réseau local : un clic sur « Afficher l'image » peut faire émettre une requête vers un hôte interne. La réponse n'est montrée que si c'est une image.
- **Socket UDP de la webview.** Sous Windows, un processus WebView2 tient un socket UDP sur un port éphémère, toutes interfaces. C'est le comportement du moteur Chromium, pas un service d'Escouade.
- **Flux périodiques sans réglage.** Le contrôle de mise à jour (5 minutes), le catalogue du store (6 heures), `git fetch` (5 minutes) et la lecture des quotas (5 minutes, par un process de Claude Code) n'ont pas de réglage pour être coupés. Un réglage « mode hors ligne » simplifierait le déploiement en entreprise.
- **Archives de plugins.** Le lecteur d'archives refuse les liens, les chemins absolus et les `..` ; il accepte un nom de fichier contenant `:` ou un nom réservé de Windows, sans conséquence constatée.
- **Programmes en parallèle.** La limite de 4 programmes à la fois par plugin n'a pas pu être observée en boîte noire (le programme d'essai se termine trop vite) ; un test du projet la couvre.
- **Code de test des crates.** Ce qui ne sert qu'aux tests (secrets gardés en mémoire, faux services, programmes des harness non cherchés) est derrière une fonction de compilation, `testing`. Elle n'entre que par des dépendances de développement : `cargo tree -e no-dev` ne la montre sur aucune crate d'une construction normale. Une dépendance ordinaire qui l'activerait la ferait entrer dans le binaire publié sans que rien le signale.
- **Cache des modules précompilés des plugins.** L'hôte garde dans le dossier de données une copie précompilée de chaque module de plugin, et la recharge comme du code natif, sur la foi du fichier (`crates/escouade-plugins/src/wasm.rs:355-367`). Aucune empreinte de cette copie n'est gardée ailleurs : un programme de la session qui écrit dans ce dossier ferait exécuter son code par l'application, hors du bac à sable. Il a déjà les droits de l'utilisateur ; un plugin, lui, n'écrit pas dans ce dossier (W-08).
- **Secrets Apple dans le job de publication.** Sous macOS, le workflow de publication pose les secrets de signature Apple dans l'environnement du job, à la dernière étape avant la construction (`.github/workflows/release.yml:251-267`). Les étapes qui la suivent les voient : la construction qui signe, les contrôles du dépôt, et les étapes de clôture des actions du job (le cache, Node, la copie du dépôt), toutes figées par empreinte. Les secrets Azure et la clé des mises à jour, eux, ne sont donnés qu'à l'étape qui signe.

Preuve. `crates/escouade-plugins/src/files.rs:590-619` (les noms réservés ne sont pas écartés), `src/components/conv/Markdown.svelte:38-64` (image d'un chemin local), `src-tauri/src/sentfiles.rs:594-625` (image distante), `src/lib/updater.ts:13-14`, `crates/escouade-plugins/src/store.rs:30`, `src-tauri/src/core.rs:751-752`, `src-tauri/src/usage.rs:451` (les quatre périodes), `crates/escouade-plugins/src/store.rs:1360-1375` (noms d'une archive), `crates/escouade-plugins/src/procs.rs:39`, `src-tauri/Cargo.toml:240-253` (la fonction `testing`, dans les dépendances de développement), `crates/escouade-plugins/src/wasm.rs:240-246` (le cache des modules).

Scénario. Aucun scénario d'attaque direct : ce sont des points de comportement à connaître avant de régler un pare-feu et des postes.

Recommandation. Écarter les noms réservés de Windows dans `fs/*` et dans le lecteur d'archives. Refuser les adresses du réseau local au téléchargement d'une image distante. Ajouter un réglage « mode hors ligne » qui coupe d'un geste les quatre flux périodiques. Contrôler en CI que la fonction `testing` n'est sur aucune crate d'une construction de publication. Garder l'empreinte de chaque module précompilé avec celle du module, et recompiler quand elle ne concorde pas. Donner les secrets Apple à la seule étape qui signe, comme les autres.

6. Les contrôles en place

Ce chapitre montre ce qui est en place, partie par partie, avec la preuve dans le code et l'essai qui l'a mis à l'épreuve. Les identifiants d'essais renvoient à l'annexe D. Les sections 6.9 et 6.10 disent ce que le cœur vérifie de ce qu'un plugin de harness lui décrit, et ce que ces gardes laissent passer ; la section 6.11, ce qui a été contrôlé du produit publié.

6.1 Ce qui sort du poste, ce qui écoute

Contrôle	Preuve	Essai
Aucun port n'écoute hors de `127.0.0.1`	`crates/escouade-mcp/src/port.rs:69` et `crates/escouade-mcp/src/port.rs:91` (`Ipv4Addr::LOCALHOST`), `src-tauri/src/tui.rs:920` ; aucune autre liaison dans le code hors tests	B-01 : le processus n'écoute que sur `127.0.0.1`, et rien ne répond sur l'adresse réseau du poste
Les statistiques d'usage ne partent qu'après un oui, et ne portent ni nom, ni chemin, ni texte	`src-tauri/src/telemetry.rs:37`, `:355-380`, `:391-393` ; `crates/escouade-model/src/model.rs:756`	Tests du projet `telemetry::tests` (rejoués)
Une instance de test n'envoie jamais de statistiques	`src-tauri/src/telemetry.rs:323-324` (`allowed`)	—
L'application dit qui elle est aux proxys et ne se fait pas passer pour un navigateur	`crates/escouade-updates/src/net.rs:9-12`, `:116-132`	—
Un proxy qui répond par un refus n'est jamais contourné	`src-tauri/src/updates.rs:854-857`	Tests du projet `updates::net_tests` (rejoués)
Les modèles téléchargés (dictée, voix, recherche) sont vérifiés par une empreinte SHA-256 écrite dans le code	`crates/escouade-voice-local/src/models.rs:153-182`, `crates/escouade-vectors/src/model.rs:49-56`, `:275-284`	—
Une image distante n'est jamais chargée par la fenêtre : elle demande un clic, et c'est le cœur qui la lit, sans cookie ni référent	`src/lib/markdown.ts:27-46`, `src-tauri/src/sentfiles.rs:594-625`	B-06 : aucune requête vers le serveur leurre
Le cœur n'appelle pas le fournisseur de modèles : le quota de Claude Code est lu par Claude Code lui-même, lancé sans tour ni session	`src-tauri/src/core.rs:10969-11021`, `plugins/claude/src/terminal.rs:116-136`	Recherche dans le code hors tests : aucune adresse d'Anthropic ; tests du projet
Derrière un proxy qui inspecte le TLS, un fichier d'autorités de certification suffit ; il est donné aussi aux agents	`crates/escouade-plugins/src/tls.rs:170-184`	Tests du projet
« Ignorer la vérification des certificats TLS » ne vaut que pour une connexion qui passe par le proxy, et l'interface l'affiche dans un bandeau : sans proxy, ou pour un hôte exclu du proxy, le certificat est vérifié	`crates/escouade-plugins/src/tls.rs:263-284`	W-27 : onze écritures d'une exclusion reconnues, onze exclusions d'un autre hôte sans effet ; tests du projet
Une seule lecture des exclusions du proxy décide de la route d'une connexion et de la vérification de son certificat	`crates/escouade-plugins/src/tls.rs:49-106`	W-28, sur le fil : un serveur local exclu par douze écritures est joint directement et refusé pour son certificat, douze fois sur douze ; non exclu, il passe par le proxy

6.2 Les secrets

Contrôle	Preuve	Essai
Les jetons des comptes et des plugins sont au trousseau du système (Windows, macOS), jamais dans le stockage en mémoire de repli	`crates/escouade-forge/src/integrations/secrets.rs:54-96`	—
Un fichier qui tient un secret est écrit en mode privé, sur disque avant de remplacer l'ancien	`crates/escouade-model/src/paths.rs:688-713`	Tests du projet `integrations::secrets` (rejoués)
Un plugin n'obtient que ses propres secrets, jamais ceux d'un compte, d'un harness ou d'un autre plugin	Hôte des plugins, méthode `secrets/get`	B-09 : 5 demandes hors de son droit, 5 refus
Le journal d'un plugin masque ses secrets	—	B-09 : le plugin écrit son secret dans son journal ; le journal montre `••••`, et aucun fichier du dossier de données ne contient la valeur
Le cœur lit la connexion de Claude Code pour savoir si un compte est connecté, n'en garde qu'une empreinte tronquée, et ne s'en sert pour aucune requête	`crates/escouade-engines/src/signin.rs:32-40`, `:53-58`, `:93-103`	Recherche dans le code hors tests : aucune adresse d'Anthropic ; tests du projet
Le jeton d'Escouade ne peut pas être mis par un plugin dans les arguments d'un harness	`src-tauri/src/plugins/harness.rs:1103-1109`	W-05
Les consignes passées en argument ne vont pas au journal : seule leur longueur y figure	`src-tauri/src/plugins/harness.rs:1342-1355`	Journal de l'instance de test relu
Les erreurs du réseau des plugins ne citent jamais la requête d'une adresse, qui peut porter une clé	`crates/escouade-plugins/src/wasm.rs:1519-1521`	Test du projet `a_failure_never_says_the_query_of_its_url` (rejoué)
Copilot CLI reçoit le jeton du serveur MCP par un fichier privé, pas dans une variable d'environnement dont ses commandes hériteraient	`plugins/copilot/src/terminal.rs:230-236`	Lu

6.3 Les agents, le serveur MCP local, les hooks, Remote Control

Contrôle	Preuve	Essai
Le serveur MCP est éteint par défaut	`crates/escouade-model/src/model.rs:745`	—
Chaque requête passe quatre gardes avant d'atteindre la bibliothèque MCP : `Host`, absence d'`Origin`, jeton, chemin	`crates/escouade-mcp/src/http.rs:264-300`	B-02 : 41 requêtes forgées, 41 refus
Une page web ne peut pas appeler le serveur, même par DNS rebinding : tout `Origin` est refusé, tout `Host` étranger aussi, et aucun en-tête CORS n'est renvoyé	`crates/escouade-mcp/src/http.rs:279-291`, `:336-342`	B-02 : 7 `Host` forgés, 5 `Origin`, une pré-requête CORS
Jetons de 256 bits tirés du générateur du système, comparés en temps constant, tous à chaque fois	`crates/escouade-mcp/src/lib.rs:57-83`, `:85-90`	B-02 : jeton tronqué, allongé, dans l'URL, dans un cookie : refusés

Contrôle	Preuve	Essai
Un jeton par agent, en mémoire, retiré à l'arrêt de l'agent ; éteindre l'option change le jeton à l'instant	`src-tauri/src/mcp/mod.rs:267-280`, `:360-393`, `:396-407`	Tests du projet `mcp::tests` (rejoués)
Sans jeton, rien ne distingue un chemin d'un autre	`crates/escouade-mcp/src/http.rs:286-298`	B-02 : 8 chemins, 401 partout sans jeton
Un agent ne répond pas à sa propre demande, ne s'écrit pas, ne s'arrête pas lui-même	`src-tauri/src/mcp/act.rs:1760-1790`	B-04
« Toujours autoriser » et le mode `bypassPermissions` restent des décisions de l'utilisateur, dans la fenêtre	`src-tauri/src/mcp/act.rs:874-890`, `:1941-1946`	B-04
Un agent ne joint à un ticket que des fichiers de son dossier de travail	Outil `create_ticket`	B-04
Un projet privé n'est vu que par ses propres agents	`src-tauri/src/mcp/act.rs:113-135`	Tests du projet `mcp::private_tests` (rejoués)
Aucun outil MCP n'allume Remote Control	`src-tauri/src/mcp/tools.rs:226`	—
Le serveur de hooks n'écoute que sur la boucle locale, contrôle `Host`, `Origin` et jeton, et ne décide rien : il répond toujours `{}`	`src-tauri/src/tui.rs:2834-2881`	—
Le programme `escouade-hook` n'envoie qu'à la boucle locale, et sort sans rien faire s'il n'a pas de jeton	`src-tauri/src/bin/escouade-hook.rs:29-43`, `:66-78`	Tests du programme (dans le dépôt)
Les commandes d'un fichier `.escouade.yml` venu d'un dépôt ne tournent qu'après accord de l'utilisateur sur leur empreinte	`src-tauri/src/projectfile.rs:1044-1058`, `src-tauri/src/commands.rs:1311-1314`	W-21 (fuzz du lecteur)
Le lien `escouade://` n'ouvre que la page d'un plugin par un identifiant bien formé, et n'installe jamais rien	`src-tauri/src/deeplink.rs:28-50`	W-16 : 14 liens forgés refusés, 55,5 millions d'entrées générées
Un agent ne répond ni aux demandes de permission ni aux questions d'un autre agent : un réglage à trois choix, « Personne » par défaut, qu'aucun outil du serveur ne change	`src-tauri/src/mcp/act.rs:1903-1929`	B-04 : refusé par les quatre voies essayées ; aucun des 59 outils d'un agent ne touche à ce réglage ; tests du projet
Un `tools/call` au nom inconnu est refusé avant la bibliothèque du serveur ; 2 Mio au plus pour une requête sans pièce jointe	`crates/escouade-mcp/src/http.rs:314-336`	B-05 : 2 800 requêtes aux noms de 256 Kio, dont 1 800 par neuf formes détournées : toutes refusées, mémoire de l'application stable
Le mode sans aucune demande (« Bypass ») n'existe que sur un réglage, éteint par défaut : sans lui, aucun agent ne démarre ni ne passe dans ce mode, et les arguments connus qui l'ouvrent sont retirés de tout lancement	`crates/escouade-model/src/model.rs:53`, `:660` ; `src-tauri/src/bypass.rs:17-86`	W-26 : les 18 formes connues retirées (limite : section 6.10) ; B-04 ; tests du projet
Le mode des agents se choisit au premier démarrage, sans « Bypass » parmi les choix	`src/lib/first-mode.ts:5-7`	Lu ; test du projet
Un harness tiers n'entre dans un projet privé qu'activé par l'utilisateur ; une vue de terminal qui ne tient pas la clôture est refusée à un agent clôturé	`src-tauri/src/harness_scope.rs:250-253`, `src-tauri/src/plugins/harness.rs:1111-1131`	W-29 ; tests du projet

Contrôle	Preuve	Essai
`escouade-hook` est appelé par son chemin entier ; quand une ligne ne peut pas le porter (un espace, un accent), par son nom, son dossier en tête du `PATH` et la recherche dans le dossier courant coupée	`crates/escouade-acp/src/terminal.rs:60-126`, `plugins/copilot/src/terminal.rs:68-83`	W-30 : dix chemins, puis `cmd` lancé dans un dossier qui tient un leurre

6.4 Les plugins

Contrôle	Preuve	Essai
Un plugin tourne dans un bac à sable WebAssembly (wasmtime) : pas de dossier, pas de socket, seulement les trois flux standard	`src-tauri/Cargo.toml` (dépendance `wasmtime`, WASI preview 1), `crates/escouade-plugins/src/wasm.rs:1-34`	B-09 : descripteurs 3 et 5 refusés, lecture de fichier hors dossier refusée
Mémoire bornée (64 Mio, 256 au plus) et tour de calcul borné (2 s, 10 au plus) ; l'instance fautive est arrêtée, puis n'est plus relancée après 4 arrêts en 5 minutes	`crates/escouade-plugins/src/wasm.rs:89-134`	B-10 (mémoire), B-11 (boucle) : l'application reste disponible
Réseau : https seulement, hôtes déclarés, aucune adresse écrite, aucune redirection suivie	`crates/escouade-plugins/src/wasm.rs:1125-1183`, `:1243`	W-09 : 24 adresses piégées refusées ; B-09 : 15 tentatives refusées
Jamais une adresse de la machine ou du réseau local, jugée à la connexion sur ce que le nom résout	`crates/escouade-plugins/src/wasm.rs:805-845`, `:880-924`	W-10 : 26 adresses interdites ; réponse de type DNS rebinding : la machine n'est jamais jointe
Fichiers : seulement les racines déclarées, liens suivis avant de juger, clôture des projets privés tenue par l'hôte	`crates/escouade-plugins/src/files.rs:397-423`, `:590-619`	W-08 : `..`, chemins absolus, UNC, flux NTFS, jonction sortante, autre casse : refusés ; B-09 : 16 tentatives refusées
Programmes : seulement ceux de la permission, mot pour mot, trouvés dans le `PATH`, jamais un chemin ni un shell	`crates/escouade-plugins/src/procs.rs:96-144`, `crates/escouade-plugin-cli/src/manifest.rs:795-807`	W-07 ; B-09 : 10 tentatives refusées
Les variables d'Escouade ne sont pas transmises aux programmes d'un plugin	`crates/escouade-plugins/src/procs.rs:81-93`	W-06
Le cœur ne laisse jamais un plugin nommer le programme d'un harness : il le trouve lui-même	`crates/escouade-plugin-protocol/src/engine.rs:203-207`	W-01
Les fichiers d'un lancement restent dans le dossier de l'agent (chemin lu tel qu'écrit : voir SEC-06)	`src-tauri/src/plugins/harness.rs:1246-1260`	W-01 : 14 chemins sortants refusés
L'identifiant `dev.escouade.*` est refusé à un tiers	`crates/escouade-plugin-cli/src/manifest.rs:74-80`	W-17 : 4,6 millions de manifestes générés, aucun tiers n'obtient cet identifiant ni une commande par chemin
Sceau d'une version : signature minisign vérifiée sur le poste avec les clés embarquées, liée à l'identifiant et à la version	`crates/escouade-plugins/src/store_keys.rs:16-19`, `:115-134`	W-13 : 1 721 altérations d'un bit ou d'un caractère, aucune acceptée
Ce qui s'installe est ce que le store a nommé : `version.json`, archive, manifeste et module, chacun par son SHA-256	`crates/escouade-plugins/src/store.rs:772`, `:881-986`	W-13 ; tests du projet `plugins_store_tests` (rejoués)

Contrôle	Preuve	Essai
Une clé révoquée ne scelle plus ; une liste de révocation non signée, signée par une clé révoquée ou plus ancienne est refusée	`crates/escouade-plugins/src/store.rs:284-335`	W-13, W-20
Une archive est lue en mémoire : ni lien, ni chemin sortant, ni bombe de décompression	`crates/escouade-plugins/src/store.rs:1314-1392`	W-18 : 8 archives piégées refusées, 1,9 million d'entrées générées
Une version installée dont les fichiers ont changé est chargée désactivée	`crates/escouade-plugins/src/lib.rs:940-942`	Tests du projet `plugins_tests`
Aucune mise à jour de plugin ne s'installe seule	`crates/escouade-plugins/src/lib.rs:446`	—
Les clés de test du store ne sont lues que par un binaire de développement	`crates/escouade-plugins/src/store_keys.rs:37-52`	—
Un plugin ne pose sur ses programmes et sur son harness que les variables d'environnement que son manifeste déclare, montrées à l'installation ; jamais celles des familles par lesquelles une commande en devient une autre (configuration de git, chargeur, proxy, autorités)	`crates/escouade-plugin-cli/src/manifest.rs:475-518`, `src-tauri/src/plugins/harness.rs:1156-1171`	W-06, W-07 : les 30 variables essayées refusées non déclarées, 28 refusées même déclarées ; W-22 : 27 autres graphies refusées ; B-09
Le harness qu'un plugin tiers décrit n'est ni un chemin, ni un shell, ni un interpréteur connu ; sa version n'est lue qu'après le démarrage d'un agent ou d'un terminal de ce harness	`crates/escouade-plugin-cli/src/manifest.rs:526-569`, `src-tauri/src/engine/registry.rs:413-430`	W-25 : 31 noms refusés (limite : SEC-01) ; tests du projet
Ce que le manifeste d'un harness tiers fait dire à l'application (les commandes qui l'installent et le mettent à jour) est montré à l'installation, et dit « donné par le plugin » là où il s'affiche ; les variables qu'il retire d'un lancement sont jugées comme celles qu'il pose	`crates/escouade-plugins/src/described.rs:56-75`, `src-tauri/src/engine/registry.rs:376-384`, `src-tauri/src/plugins/harness.rs:1176-1192`	W-32 ; tests du projet (`src-tauri/src/plugins_acp_tests.rs:1523`, `:1570`, `:1614`, `:1657`)
« N'installer que des versions vérifiées » est allumé pour une installation neuve	`crates/escouade-model/src/model.rs:769`	Test du projet ; B-15 pour sa portée (SEC-01)
La main de l'éditeur vient de l'origine d'un plugin et de son identifiant, ensemble : un dossier chargé en développement, copié ou cloné est toujours lu comme un tiers	`crates/escouade-plugins/src/lib.rs:221-233`, `:1343-1349`	W-31 ; B-14 : trois dossiers, sous un identifiant tiers ou `dev.escouade.*`, refusés à la lecture du manifeste ; test du projet
« Tout mettre à jour » ne confirme ensemble que des versions vérifiées qui n'ajoutent rien : une version que personne n'a relue a sa propre fenêtre, avec son avertissement	`src/lib/plugins-view.svelte.ts:554-559`, `src/components/plugins/UpdateAllModal.svelte:39`	B-16 : aucune fenêtre commune ne la porte ; fermée sans accord, rien n'est installé ; tests du projet (`src/components/plugins/plugins.test.ts:735`, `:749`, `:790`)

6.5 La chaîne d'approvisionnement

Contrôle	Preuve	Essai
Mises à jour signées (minisign), clé publique dans le binaire	`src-tauri/tauri.conf.json:72`	B-13 : signature absente, mal formée, d'une autre clé, d'autres octets : rien ne s'installe

Contrôle	Preuve	Essai
Jamais de retour à une version plus ancienne, jamais de bêta sans l'avoir demandée	`src-tauri/src/updates.rs:754-756`	B-13 : version plus ancienne, égale, bêta : non proposées
L'adresse des mises à jour ne se change par variable que dans un binaire de développement	`src-tauri/src/updates.rs:925-931`	—
Binaires Windows signés par le service de signature d'Azure ; binaires macOS signés et notariés, vérifiés par `codesign --verify --strict` avant publication	`.github/workflows/release.yml:224-240`, `:249-277`, `:302-327`	Sur les fichiers publiés : P-02 (Authenticode valide) ; P-03 (macOS : signature et ticket présents, notarisation lue dans le journal du job)
Dépendances verrouillées : `Cargo.lock` et `package-lock.json` versionnés, `npm ci` et `--locked` en CI	`.github/workflows/release.yml:211`	—
Toutes les crates viennent de crates.io ; licences compatibles	—	`cargo deny` : sources ok, licences ok, interdictions ok (annexe A)
Outil de compilation des plugins épinglé par version et par SHA-256	`scripts/escouade-js.sh`, `crates/escouade-plugin-cli/src/wasm.rs:231-250`	—
Le SDK des plugins est publié avec une identité de workflow (`id-token`), sans jeton durable	`.github/workflows/sdk-publish.yml:33-35`	—
Peu de code `unsafe` de première main, concentré dans les appels au système (processus, pseudo-terminaux, WSL)	`crates/escouade-model/src/job.rs`, `pty.rs`, `proc.rs`, `wsl.rs`	`cargo geiger` (annexe C)
Les avis de sécurité des dépendances sont contrôlés à chaque pull request, à chaque poussée et chaque nuit, de façon bloquante pour l'application : aucun n'y est atteignable. Dependabot propose les mises à jour chaque mois	`.github/workflows/advisories.yml:13`, `:47`, `:76` ; `.github/dependabot.yml:7-87`	`cargo audit`, `cargo deny`, `npm audit` relancés (annexes A et B) ; ce qui reste ouvert hors de l'application : SEC-04
Les actions de la CI sont figées par l'empreinte de leur commit (83 références dans sept workflows, aucune étiquette). Le workflow de publication est en lecture seule par défaut, et les secrets de signature sont donnés à l'étape qui signe ; ceux d'Apple, à partir de la dernière étape avant la construction (SEC-18)	`.github/workflows/release.yml:17-18`, `:234-236`, `:281-287`, `:539-542`	Lu ; `semgrep` ne relève aucune action référencée par une étiquette (annexe C)
Chaque binaire publié porte une attestation de provenance, que le workflow vérifie lui-même comme un utilisateur le ferait	`.github/workflows/release.yml:549-592`	P-04 : vérifiée pour les cinq binaires de la 2.2.0 (section 6.11)
Le code qui ne sert qu'aux tests n'entre pas dans une construction normale de l'application	`src-tauri/Cargo.toml:240-253`	`cargo tree -e no-dev` : la fonction `testing` n'est sur aucune crate (SEC-18)

Reproductibilité. Les modules WebAssembly des plugins sont compilés de façon déterministe par le store (même empreinte sur tous les systèmes, d'après le code de `escouade-plugin build`). Les binaires de l'application, eux, ne sont pas reproductibles : aucun dispositif ne permet à un tiers de reconstruire le même installateur et de comparer. Chaque binaire publié porte une attestation de provenance, qui le relie à son commit et à son workflow (section 6.11) : elle ne rend pas la construction reproductible.

6.6 La webview

Contrôle	Preuve	Essai
Le HTML brut d'un message est affiché comme du texte ; le reste passe par DOMPurify, sans `style`, `form` ni `input`	`src/lib/markdown.ts:22-26`, `:78-82`	B-06 : 32 charges, aucun script exécuté, aucun élément actif dans la page
Politique de contenu : pas de script en ligne ni distant, pas d'`eval`, pas de connexion sortante, pas d'image, de média ou de cadre distants	`src-tauri/tauri.conf.json:32`	B-07 : 15 violations bloquées, aucune requête reçue par le serveur leurre
La fenêtre ne suit jamais un lien : seuls `http(s)` et `mailto` s'ouvrent, dans le navigateur	`src/lib/links.ts:19-30`, `:44-62`	B-06 : liens `javascript:`, `data:`, `file:`, UNC, `ms-msdt:`, `escouade:` neutralisés
La page de l'application en cours de développement est dans une webview à part : aucune commande ne lui répond, son stockage est séparé	`src-tauri/src/windowthread.rs:224-231`, `src-tauri/src/devpage.rs:6-15`, `src-tauri/capabilities/default.json:5`	Tests du projet `devpage_tests` (rejoués). Cette webview peut aussi s'afficher dans un volet de l'écran partagé : c'est la même, avec les mêmes droits
Les outils de développement de la fenêtre sont fermés dans une version publiée	`src-tauri/src/lib.rs:410-412`	—
Le README d'un plugin est assaini et ses images sont copiées par le store, pas chargées depuis un tiers	`docs/STORE.md`, §7	—
La bibliothèque d'assainissement (`dompurify` 3.4.17) est à jour : aucun avis de sécurité ne vise le code livré de l'interface	`package-lock.json:2477-2478`	`npm audit --omit=dev` : 0 (annexe B)

6.7 Les données sur le disque

Contrôle	Preuve	Essai
Écritures atomiques : un fichier d'état n'est jamais laissé à moitié écrit	`src-tauri/src/paths.rs` (`write_atomic`, `write_private`)	—
Les fichiers de jetons des agents sont effacés à l'arrêt de l'agent et au démarrage de l'application	`src-tauri/src/mcp/mod.rs:396-446`, `:513-516`	B-04 : les seuls fichiers qui contiennent un jeton sont ceux prévus pour
Refuser les statistiques supprime l'identifiant d'installation	`src-tauri/src/telemetry.rs:370-378`	—
Désinstaller un plugin efface son dossier, ses réglages et ses secrets	`docs/PLUGINS-DESIGN.md`, §3.4	B-09 (désinstallation en fin d'essai)

6.8 Le store et le site

Contrôle	Preuve	Essai
Le store ne détient aucune clé de signature : il ne peut pas fabriquer un sceau	`crates/escouade-plugins/src/store_keys.rs:1-9`	—
Le store répond avec HSTS, une politique de contenu à nonce, `nosniff`, `X-Frame-Options: DENY`, des politiques d'origine croisée	—	N-01, N-02

Contrôle	Preuve	Essai
Aucun en-tête CORS pour une origine étrangère ; `http` redirigé vers `https` ; TLS 1.0 et 1.1 refusés	—	N-04, N-05, N-06
Le store ne publie que depuis un dépôt public, fige le commit et l'archive, et n'exécute jamais un plugin hors d'un conteneur sans réseau	`docs/STORE.md` , §3 et §7	Non essayé (hors périmètre : lecture seule)
Une copie de la liste de révocation est servie par un second hôte	`crates/escouade-plugins/src/store.rs:27` , `website/nginx.conf`	N-03

6.9 Ce qu'un plugin de harness décrit, et ce que le cœur vérifie

Chaque harness de l'application est servi par un plugin de première main (`dev.escouade.*` , signé). Le plugin décrit trois choses, dont le cœur ne garde pas de copie : le lancement d'un agent (`engine/launch`), celui de son interface dans un terminal (`terminal/launch`), et les lectures de son compte par un process (`account/read` : ses modèles, son quota, ce que l'interface doit lire avant de démarrer). Le cœur trouve lui-même le programme à lancer, et contrôle ce qu'on lui décrit avant de rien écrire ni lancer.

Un plugin tiers ne sert aucune de ces demandes : elles ne vont qu'aux plugins que l'application nomme elle-même (`src-tauri/src/plugins/harness.rs:300-313` , `:458-466`). Le harness d'un tiers est décrit par son manifeste, sans code, et servi par le plugin ACP de l'éditeur (`:596-621`).

Ce que le plugin décrit	Ce que le cœur vérifie	Preuve	Essai
Les arguments	Le jeton d'Escouade n'y figure pas. Ceux qui ouvrent le mode sans aucune demande sont retirés tant que le réglage est éteint, et toujours pour une lecture de compte	`src-tauri/src/plugins/harness.rs:1103-1110` , `:774` , `:863`	W-05, W-26, W-29
Les variables posées	Déclarées par le manifeste du plugin ; pour un tiers, jamais d'une famille refusée	`src-tauri/src/plugins/harness.rs:1156-1171` , `:1195-1201`	W-06, W-07, W-22, W-24
Les variables retirées	Pour un tiers : aucune d'une famille refusée, ni les variables d'autorités, ni celles que le cœur donne. Une variable d'aucune famille refusée se retire (liste de refus)	`src-tauri/src/plugins/harness.rs:1176-1192`	W-32 ; tests du projet
Les fichiers à écrire	Sous le dossier de l'agent, ou du compte pour une lecture ; ou à côté du fichier privé de l'interface. Le chemin est jugé tel qu'il est écrit (SEC-06)	`src-tauri/src/plugins/harness.rs:1246-1292`	W-01, W-02, W-03
La clôture d'un projet privé	Chaque chemin de la clôture figure dans le lancement, ou le plugin de l'éditeur dit la tenir lui-même. Le contrôle cherche une sous-chaîne (SEC-02)	`src-tauri/src/plugins/harness.rs:1111-1131`	W-04, W-29
La lecture seule d'une discussion	Le lancement doit la déclarer	`src-tauri/src/plugins/harness.rs:1132-1137`	Tests du projet
Les arguments qu'une lecture « avant » rend pour l'interface	Rejugés comme ceux du lancement	`src-tauri/src/plugins/harness.rs:811-824`	Tests du projet
La relance d'un agent, l'attache d'un terminal à un process partagé	Jugées comme un premier lancement	`src-tauri/src/plugins/harness.rs:1206-1227`	Tests du projet

Ce que le plugin décrit	Ce que le cœur vérifie	Preuve	Essai
Une lecture de compte	60 secondes au plus ; le process est arrêté ensuite	`src-tauri/src/plugins/harness.rs:953-963`	—

Ce que le cœur ne juge pas : la longueur d'une ligne lue d'un process (SEC-08), et les arguments eux-mêmes hors les deux règles ci-dessus, puisque ce sont ceux du harness. Ces contrôles sont une seconde ligne. La première est le sceau du plugin : c'est lui qui dit qui a écrit ces descriptions (SEC-03).

6.10 Ce que les gardes tiennent, et ce qu'elles laissent passer

Plusieurs des contrôles de ce chapitre sont des gardes écrites comme des règles ou des listes. Chacune a été éprouvée par les voies qu'un attaquant essaierait d'abord : une autre écriture du même nom, une autre forme du même argument, un chemin détourné. Cinq d'entre elles sont des **listes de refus** : elles ferment ce que leurs auteurs ont su nommer, pas tout ce qui est imaginable. Le lecteur doit le savoir.

Garde	Ce qui a été essayé contre elle	Résultat
Réponse aux demandes des autres agents	Autoriser, refuser, répondre comme à une question, réglage « chef » sans chef ; recherche d'un outil qui désigne le chef ou change un réglage	**Tient.** Aucun des 59 outils d'un agent ne touche à ce réglage
Porte du serveur MCP	Lot, clés écrites deux fois, octets après le JSON, marque d'ordre des octets, échappement, autre casse : 1 800 requêtes	**Tient.** Mémoire inchangée
Variables d'un plugin	27 graphies d'une variable refusée ; variable déclarée dans une casse, posée dans une autre	**Tient** (W-22)
	29 variables qui ne sont d'aucune famille refusée : `SSH_ASKPASS`, `LESSOPEN`, `RUSTC_WRAPPER`, `DOTNET_STARTUP_HOOKS`, `OPENSSL_CONF` …	**Passent une fois déclarées** (W-24). La déclaration, affichée à l'installation, devient le contrôle
	Dossier en développement sous un identifiant de l'éditeur	**Refusé** (B-14, W-31) : un dossier en développement est toujours lu comme un tiers
Interpréteurs refusés à un harness tiers	31 noms de shells et d'interpréteurs, avec ou sans `.exe`, en toute casse	**Tient**
	49 autres programmes qui exécutent ce que leurs arguments disent : `py`, `pythonw`, `pwsh-preview`, `npx`, `uvx`, `dotnet`, `java`, `make` …	**Passent tous** (W-25) : SEC-01. L'écran d'installation montre la ligne de commande
« Versions vérifiées » par défaut	Réglage allumé : installation depuis un dossier, puis en développement	**Passent** (B-15) : le réglage ne juge que le store (SEC-01)
Tri de « Tout mettre à jour » par le sceau	Une mise à jour non vérifiée qui ne demande rien de plus, parmi d'autres mises à jour	**Tient** (B-16) : aucune fenêtre commune ne la porte, elle a la sienne, avec son avertissement
Variables retirées par un harness tiers	Cinq variables de proxy et d'autorités au manifeste ; 18 variables d'une famille refusée ou données par le cœur, en plusieurs casses, dans un lancement	**Tient** (W-32) : refusées au manifeste, gardées au lancement

Garde	Ce qui a été essayé contre elle	Résultat
	10 variables d'aucune famille refusée : `AWS_CA_BUNDLE`, `DENO_CERT`, `PIP_CERT`, `CARGO_HTTP_CAINFO`…	**Se retirent toutes** (W-32). Le manifeste les montre à l'installation
Retrait des arguments du mode Bypass	Les 18 formes connues, seules ou parmi d'autres arguments	**Tient** : toutes retirées
	12 autres écritures : formes courtes (`-y`, `-f`), options équivalentes de Codex, configuration par arguments, réglages en JSON, autre casse, abréviation	**Passent toutes** (W-26). Théorique : ce que chacune ouvre dépend du harness, et aucun harness réel n'a été lancé. La garde première, le refus du mode, n'en dépend pas
« Ignorer la vérification » bornée au proxy	Sans proxy ; hôte exclu par son nom ou son adresse	**Tient** (tests du projet, W-27)
	Hôte exclu par une plage d'adresses	**Tient** (W-28), pour douze écritures d'une exclusion
Refus par chemin (clôture, fichiers d'un lancement)	Nom court de Windows, point ou espace final, flux du répertoire, autre casse, `..` : 13 graphies d'un dossier clôturé, 7 d'un dossier d'agent	**Tient** (W-23, W-01, W-08)
Refus d'écriture de la vue Copilot CLI	La vue demandée à son plugin comme le cœur la demande : mode sans demande, discussion, agent clôturé	**Tient pour ce que le cœur et le plugin écrivent** (W-29) : les refus sont là, une discussion refuse tout, un agent clôturé n'a pas la vue. **Non éprouvé** au-delà : leur tenue face à une autre écriture du même dossier dépend de Copilot CLI, qui n'a pas été lancé. En mode Bypass, aucun refus ne nomme une commande : une commande écrit là où une écriture est refusée
Hooks d'une vue appelés hors du dossier de l'agent	Dix chemins d'installation qu'une ligne de commande ne peut pas porter ; `cmd` lancé comme un hook dans un dossier qui tient un leurre du même nom	**Tient** (W-30) : le vrai programme est lancé. Sans la variable que le lancement pose, c'est le leurre qui l'est

Trois remarques qui valent pour toutes ces gardes :

- **Elles sont dans un fichier de réglages du profil.** Sans projet privé, rien n'interdit à un agent d'écrire ce fichier (`src-tauri/src/fence.rs:13`) ; l'application le relit à son démarrage. Un agent à qui une écriture hors de son dossier est accordée peut donc rallumer une option pour le démarrage suivant. C'est la limite de tout outil qui lance des agents avec les droits de l'utilisateur : le mode « Demander » fait passer cette écriture devant l'utilisateur.
- **Une installation mise à jour garde ses anciens choix** : mode `auto`, mode Bypass s'il servait, plugins non vérifiés. Les défauts stricts de la 2.2.0 ne valent que pour une installation neuve.
- **Aucun harness réel n'a été lancé.** Ce que Claude Code, Codex ou Copilot CLI font d'un argument ou d'une règle de refus est supposé conforme à leur documentation.

6.11 Le produit publié

Les fichiers de la 2.2.0, publiée le 14 novembre 2026 à 10 h 49 UTC, ont été téléchargés depuis sa page dans un dossier hors du dépôt, et contrôlés sous Windows sans être lancés : l'installateur Windows, l'archive de mise à jour et l'image disque macOS, l'AppImage, le paquet deb et le manifeste de mise à jour. Le run de publication (43583704479) a réussi tous ses jobs.

Id	Contrôle	Résultat
P-01	Signature de mise à jour, lue comme l'application la lit : le manifeste de la version (9 plateformes) et la clé publique embarquée, par `tools/verify-update-signature.mjs`	**Valide** pour les quatre paquets (Windows, macOS, AppImage, deb). Chaque signature nomme la clé embarquée ; une copie altérée d'un octet est refusée
P-02	Signature Authenticode de l'installateur Windows, lue par Windows	**Valide**, horodatée, chaîne reconnue par le système
P-03	Paquet macOS, lu par `tools/inspect-macos-package.mjs`, et journal du job macOS du run de publication	**Présents** dans le paquet : le sceau, une signature Developer ID horodatée sur les deux architectures, le durcissement d'exécution, un ticket de notarisation agrafé. Le journal dit « Notarizing Finished with status Accepted », puis `spctl` : « accepted, source=Notarized Developer ID ». Lu dans le journal, non rejoué sur un Mac
P-04	Attestation de provenance des cinq binaires, par `tools/verify-provenance.mjs` (`gh attestation verify`)	**Vérifiée** pour chacun : elle nomme le fichier par son empreinte SHA-256, le workflow `release.yml` à l'étiquette `v2.2.0`, le commit `ab351a00` de l'étiquette, un exécuteur hébergé par GitHub ; elle est inscrite au journal de transparence de Sigstore

Les empreintes et le détail des contrôles sont dans `annexes/essais/` (`signatures-mise-a-jour.txt`, `paquet-macos.txt`, `produit-publie.txt`, `attestation-de-provenance.txt`).

Ce que ces contrôles disent, et ce qu'ils ne disent pas. Une signature de mise à jour valide dit que le fichier est celui que l'éditeur a signé avec la clé de l'application : c'est ce qui protège une mise à jour. La signature Authenticode dit à Windows qui a publié l'installateur. L'attestation de provenance dit de quel commit et par quel workflow le binaire a été construit : c'est elle qui relie le fichier publié au code que cet audit a lu. Aucun de ces contrôles ne dit que la construction est reproductible (section 6.5).

Annexe A. Dépendances Rust : `cargo audit` et `cargo deny`

A.1 Analyse

`cargo audit` 0.23.0, base RustSec du 14 novembre 2026 (1 509 avis), 805 crates dans `src-tauri/Cargo.lock` au commit `ab351a00`. 7 vulnérabilités, 7 avertissements.

Avis	Lecture
RUSTSEC-2026-0367, -0368, -0369, -0429 (`zip` 4.6.1)	Écartés avec leur raison (`deny.toml:17-20`). `zip` 4.6.1 n'est tiré que par le module de mise à jour, le code de l'application employant `zip` 9.5 : la raison écrite est à récrire (SEC-04)
RUSTSEC-2026-0523, -0524 (`quinn-proto` 0.11.18), -0538 (`quinn-udp` 0.5.15)	Non atteignables. Ces crates sont une dépendance optionnelle de `reqwest`, pour HTTP/3, que l'application n'active pas : elles figurent dans le fichier de verrouillage sans entrer dans le graphe d'aucune cible (`cargo tree -i quinn-proto`, toutes fonctions activées, ne trouve rien)

`cargo audit` lit le fichier de verrouillage ; `cargo deny` juge ce qui est construit. De là l'écart entre les deux : les trois avis sur QUIC sortent du premier et pas du second.

Les 7 avertissements : six crates qui ne sont plus maintenues (`paste`, `proc-macro-error`, `ttf-parser`, `zbus_names`, `zvariant`, `zvariant_derive`), et `glib` 0.18.5, sous Linux, signalée pour un défaut de sûreté. Aucun n'est une vulnérabilité connue du code que l'application appelle.

L'espace de travail des plugins (183 crates) : 3 vulnérabilités, les trois avis sur QUIC, qui n'y sont pas compilés non plus.

`cargo deny` 0.20.2, avec le `deny.toml` du dépôt, celui que la CI emploie : **avis : ok** ; interdictions : ok ; licences : ok ; sources : ok, pour l'application et pour les plugins. 62 crates sont présentes en plusieurs versions.

A.2 Sortie brute de `cargo audit`, application

```
    Fetching advisory database from `https://github.com/RustSec/advisory-db.git`
      Loaded 1509 security advisories (from ~\.cargo\advisory-db)
    Updating crates.io index
    Scanning Cargo.lock for vulnerabilities (805 crate dependencies)
Crate:     quinn-proto
Version:   0.11.18
Title:     MAX_STREAM_DATA frame can open streams beyond the stream limit
Date:      2026-09-30
ID:        RUSTSEC-2026-0523
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0523
Solution:  Upgrade to ⩾0.11.19 (patched)

Crate:     quinn-proto
Version:   0.11.18
Title:     Long Retry token can panic or exhaust memory in a QUIC client
Date:      2026-09-30
ID:        RUSTSEC-2026-0524
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0524
Solution:  Upgrade to ⩾0.11.19 (patched)

Crate:     quinn-udp
Version:   0.5.15
Title:     Receive loop can spin after a truncated datagram
Date:      2026-09-30
ID:        RUSTSEC-2026-0538
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0538
Solution:  Upgrade to ⩾0.5.16, <0.6.0 OR ⩾0.6.3 (patched)

Crate:     zip
Version:   4.6.1
Title:      Insecure Preservation of SUID/SGID Bits during ZIP Extraction Leading to Potential Privilege Escalation
Date:      2026-10-09
ID:        RUSTSEC-2026-0367
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0367
Severity:  7.3 (high)
```

```
Solution:  Upgrade to ⩾9.0.0 (patched)

Crate:     zip
Version:   4.6.1
Title:     zip: `ZipArchive::extract` creates symbolic links whose attacker-controlled targets are outside the destination
directory
Date:      2026-10-09
ID:        RUSTSEC-2026-0368
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0368
Solution:  Upgrade to ⩾9.0.0 (patched)

Crate:     zip
Version:   4.6.1
Title:     ZipStreamReader::extract() and ZipArchive::extract() apply Unix permissions through symlinks without confining the
target
Date:      2026-10-10
ID:        RUSTSEC-2026-0369
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0369
Severity:  9.1 (critical)
Solution:  Upgrade to ⩾9.0.0 (patched)

Crate:     zip
Version:   4.6.1
Title:     zip: `ZipArchive::extract` aborts the process on a symlink entry with a large declared size
Date:      2026-10-22
ID:        RUSTSEC-2026-0429
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0429
Severity:  7.5 (high)
Solution:  Upgrade to ⩾9.0.0 (patched)

Crate:     paste
Version:   1.0.15
Warning:   unmaintained
Title:     paste - no longer maintained
Date:      2024-10-07
ID:        RUSTSEC-2024-0436
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0436
Solution:  No fixed upgrade is available!

Crate:     proc-macro-error
Version:   1.0.4
Warning:   unmaintained
Title:     proc-macro-error is unmaintained
Date:      2024-09-01
ID:        RUSTSEC-2024-0370
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0370
Solution:  No fixed upgrade is available!

Crate:     ttf-parser
Version:   0.25.1
Warning:   unmaintained
Title:     `ttf-parser` is unmaintained
Date:      2026-06-28
ID:        RUSTSEC-2026-0192
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0192
Solution:  No fixed upgrade is available!

Crate:     zbus_names
Version:   4.3.4
Warning:   unmaintained
Title:     `zbus_names` is unmaintained
Date:      2026-10-16
ID:        RUSTSEC-2026-0371
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0371
Solution:  No fixed upgrade is available!

Crate:     zvariant
Version:   5.15.0
Warning:   unmaintained
Title:     `zvariant` is unmaintained
Date:      2026-10-16
ID:        RUSTSEC-2026-0370
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0370
Solution:  No fixed upgrade is available!

Crate:     zvariant_derive
Version:   5.15.0
Warning:   unmaintained
Title:     `zvariant_derive` is unmaintained
Date:      2026-10-16
ID:        RUSTSEC-2026-0372
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0372
Solution:  No fixed upgrade is available!

Crate:     glib
```

```
Version:   0.18.5
Warning:   unsound
Title:     Unsoundness in `Iterator` and `DoubleEndedIterator` impls for `glib::VariantStrIter`
Date:      2024-03-30
ID:        RUSTSEC-2024-0429
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0429
Solution:  Upgrade to ⩾0.20.0 (patched)

error: 7 vulnerabilities found!
warning: 7 allowed warnings found
```

A.3 Sortie brute de `cargo audit`, plugins

```
    Fetching advisory database from `https://github.com/RustSec/advisory-db.git`
      Loaded 1509 security advisories (from ~\.cargo\advisory-db)
    Updating crates.io index
    Scanning Cargo.lock for vulnerabilities (183 crate dependencies)
Crate:     quinn-proto
Version:   0.11.18
Title:     MAX_STREAM_DATA frame can open streams beyond the stream limit
Date:      2026-09-30
ID:        RUSTSEC-2026-0523
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0523
Solution:  Upgrade to ⩾0.11.19 (patched)

Crate:     quinn-proto
Version:   0.11.18
Title:     Long Retry token can panic or exhaust memory in a QUIC client
Date:      2026-09-30
ID:        RUSTSEC-2026-0524
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0524
Solution:  Upgrade to ⩾0.11.19 (patched)

Crate:     quinn-udp
Version:   0.5.15
Title:     Receive loop can spin after a truncated datagram
Date:      2026-09-30
ID:        RUSTSEC-2026-0538
URL:       https://rustsec.org/advisories/RUSTSEC-2026-0538
Solution:  Upgrade to ⩾0.5.16, <0.6.0 OR ⩾0.6.3 (patched)

error: 3 vulnerabilities found!
```

A.4 Résumé de `cargo deny`

```
cargo deny 0.20.2 --manifest-path src-tauri/Cargo.toml --config deny.toml check   (deny.toml : celui du dépôt)

advisories ok, bans ok, licenses ok, sources ok

  62  warning[duplicate]
  11  warning[wildcard]
   4  warning[license-not-encountered]

Erreurs :
```

Annexe B. Dépendances npm : `npm audit`

B.1 Analyse

Projet	Résultat	Lecture
Application, code livré (`npm audit --omit=dev`)	**0**	C'est le contrôle bloquant de la CI
Application, avec les outils de construction et de test	2, faibles	`deepmerge`, tiré par un outil de construction, absent du binaire
Store, hors outils de développement	14 (6 faibles, 8 moyennes)	`hono`, `@hono/node-server`, `glob`, `minimatch`. Signalé par la CI sans la bloquer (SEC-04)
Site, hors outils de développement	10 (4 faibles, 6 moyennes)	`glob`, `minimatch` ; ces bibliothèques ne tournent qu'à la construction du site, servi en fichiers statiques. Signalé par la CI sans la bloquer (SEC-04)

La gravité affichée par `npm audit` est celle de la bibliothèque dans le pire usage, pas celle de son usage ici. Aucun avis élevé ni critique dans les trois projets.

B.2 Sortie brute, application

```
# npm audit report

deepmerge  ⩾4.2.0
deepmerge mergeObject() allows prototype poisoning of returned objects - https://github.com/advisories/GHSA-gcqx-w48m-w2h4
No fix available
node_modules/deepmerge
  @sveltejs/vite-plugin-svelte  *
  Depends on vulnerable versions of deepmerge
  node_modules/@sveltejs/vite-plugin-svelte

2 low severity vulnerabilities

Some issues need review, and may require choosing
a different dependency.
```

B.3 Sortie brute, store (hors outils de développement)

```
# npm audit report

@hono/node-server  2.0.0 - 2.1.5
Severity: moderate
@hono/node-server: Unhandled `ECONNRESET` on the socket during an asynchronous WebSocket upgrade crashes the process -
https://github.com/advisories/GHSA-qv5m-9pm6-hqp3
@hono/node-server: Backslash in the request path is parsed as a path separator, bypassing path-based access control at a reverse
proxy - https://github.com/advisories/GHSA-jpvv-r467-5p4w
fix available via `npm audit fix`
node_modules/@hono/node-server

deepmerge  ⩾4.2.0
deepmerge mergeObject() allows prototype poisoning of returned objects - https://github.com/advisories/GHSA-gcqx-w48m-w2h4
fix available via `npm audit fix --force`
Will install nuxt@4.1.3, which is a breaking change
node_modules/deepmerge
  @rollup/plugin-node-resolve  ⩾8.0.0
  Depends on vulnerable versions of deepmerge
  node_modules/@rollup/plugin-node-resolve
    nitropack  ⩾0.1.0
    Depends on vulnerable versions of @rollup/plugin-node-resolve
    Depends on vulnerable versions of archiver
    node_modules/nitropack
      @nuxt/nitro-server  ⩾3.20.0
      Depends on vulnerable versions of nitropack
      Depends on vulnerable versions of nuxt
      node_modules/@nuxt/nitro-server
        nuxt  3.20.0 - 3.21.11 || ⩾4.2.0
```

```
        Depends on vulnerable versions of @nuxt/nitro-server
        Depends on vulnerable versions of @nuxt/vite-builder
        Depends on vulnerable versions of @nuxt/vite-server
        node_modules/nuxt
          @nuxt/vite-builder  3.20.0 - 3.21.11 || ⩾4.2.0
          Depends on vulnerable versions of nuxt
          node_modules/@nuxt/vite-builder
          @nuxt/vite-server  ⩾4.6.0
          Depends on vulnerable versions of nuxt
          node_modules/@nuxt/vite-server

glob  9.0.0 - 14.0.1
Severity: moderate
Glob: AbortSignal cancellation bypass allows post-timeout filesystem traversal - https://github.com/advisories/GHSA-3g3m-2fjr-
6pw4
Depends on vulnerable versions of minimatch
fix available via `npm audit fix`
node_modules/glob
  @vercel/nft  0.29.2 - 1.11.1
  Depends on vulnerable versions of glob
  node_modules/@vercel/nft

hono  ⩽4.13.15
Severity: moderate
Hono: `mount()` and `app.mount()` can forward a path that bypasses middleware scoped inside the mount prefix -
https://github.com/advisories/GHSA-4m76-xfj5-7975
Hono: RegExpRouter skips wildcard middleware for some route sets and for paths with an encoded line terminator -
https://github.com/advisories/GHSA-h7mg-cq63-vhpq
Hono: `css` helper does not escape a closing tag split between the template and an interpolated value, allowing HTML injection -
https://github.com/advisories/GHSA-3gg2-335m-rxc6
Hono: `csrf()` does not check requests sent by hyperlink auditing (`text/ping`) - https://github.com/advisories/GHSA-46wx-v44x-
9hh2
Hono: `validator('form')` skips validation for `multipart/form-data` requests with extra parameters -
https://github.com/advisories/GHSA-4h7h-rv7c-7m2v
fix available via `npm audit fix`
node_modules/hono

minimatch  7.0.0 - 11.0.0
Severity: moderate
minimatch has potential denial of service via globparts amplification - https://github.com/advisories/GHSA-g3hv-75wg-m249
fix available via `npm audit fix --force`
Will install nuxt@4.1.3, which is a breaking change
node_modules/minimatch
  readdir-glob  2.0.0 - 3.0.0
  Depends on vulnerable versions of minimatch
  node_modules/readdir-glob
    archiver  ⩾8.0.0
    Depends on vulnerable versions of readdir-glob
    node_modules/archiver

14 vulnerabilities (6 low, 8 moderate)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force
```

B.4 Sortie brute, site (hors outils de développement)

```
# npm audit report

deepmerge  ⩾4.2.0
deepmerge mergeObject() allows prototype poisoning of returned objects - https://github.com/advisories/GHSA-gcqx-w48m-w2h4
fix available via `npm audit fix --force`
Will install nuxt@4.1.3, which is a breaking change
node_modules/deepmerge
  @rollup/plugin-node-resolve  ⩾8.0.0
  Depends on vulnerable versions of deepmerge
  node_modules/@rollup/plugin-node-resolve
    nitropack  ⩾0.1.0
    Depends on vulnerable versions of @rollup/plugin-node-resolve
    Depends on vulnerable versions of archiver
    node_modules/nitropack
      @nuxt/nitro-server  ⩾3.20.0
      Depends on vulnerable versions of nitropack
      node_modules/@nuxt/nitro-server
        nuxt  3.20.0 - 3.21.11 || ⩾4.2.0
        Depends on vulnerable versions of @nuxt/nitro-server
        node_modules/nuxt

glob  9.0.0 - 14.0.1
```

```
Severity: moderate
Glob: AbortSignal cancellation bypass allows post-timeout filesystem traversal - https://github.com/advisories/GHSA-3g3m-2fjr-
6pw4
Depends on vulnerable versions of minimatch
fix available via `npm audit fix`
node_modules/glob
  @vercel/nft  0.29.2 - 1.11.1
  Depends on vulnerable versions of glob
  node_modules/@vercel/nft

minimatch  7.0.0 - 11.0.0
Severity: moderate
minimatch has potential denial of service via globparts amplification - https://github.com/advisories/GHSA-g3hv-75wg-m249
fix available via `npm audit fix --force`
Will install nuxt@4.1.3, which is a breaking change
node_modules/minimatch
  readdir-glob  2.0.0 - 3.0.0
  Depends on vulnerable versions of minimatch
  node_modules/readdir-glob
    archiver  ⩾8.0.0
    Depends on vulnerable versions of readdir-glob
    node_modules/archiver

10 vulnerabilities (4 low, 6 moderate)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force
```

Annexe C. Analyse statique et dépendances directes

C.1 Code `unsafe` : `cargo geiger`

`cargo geiger` 0.13.0 sur la crate de l'application. Sur 524 crates du graphe, 266 contiennent du code `unsafe`, 199 n'en contiennent pas, 59 l'interdisent.

Crate de première main	Fonctions `unsafe`	Expressions `unsafe` utilisées	Où
`escouade-model`	1	254	Appels au système : objets Job de Windows, processus, groupes d'agents, WSL
`escouade` (le cœur)	1	141	Appels au système : pseudo-terminaux, fenêtres et pages, fichiers, notifications, mises à jour
`escouade-voice-local`	1	10	Audio et moteur de synthèse
`escouade-plugin-protocol`	0	8	Interface WebAssembly des plugins
`escouade-plugins` (l'hôte des plugins)	0	3	Le chargement d'un module précompilé depuis le cache de l'application (`crates/escouade-plugins/src/wasm.rs:355-367`) : observation, SEC-18
Les onze autres crates	0	0	—

Le code `unsafe` de première main est celui des appels au système d'exploitation, plus le chargement du cache des plugins. Aucun des lecteurs de données tierces examinés (manifestes, archives, signatures, liens, fichiers de dépôt) n'en contient.

```
cargo-geiger 0.13.0, cargo geiger --output-format Ascii, dans src-tauri

524 crates dans le graphe : 266 avec du code unsafe (!), 199 sans (?), 59 qui l'interdisent (:))

Les crates du dépôt (x/y : unsafe utilisé par la construction / trouvé dans la crate) :
Functions  Expressions  Impls  Traits  Methods
1/1        141/196      0/0    0/0     0/0      !  escouade 2.2.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-acp 0.1.6
0/0        0/0          0/0    0/0     0/0      ?  escouade-base 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-engines 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-forge 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-mcp 0.1.0
1/1        254/254      2/2    0/0     0/0      !  escouade-model 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-plan 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-plugin-cli 2.2.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-plugin-kit 0.1.2
0/0        8/8          0/0    0/0     0/0      !  escouade-plugin-protocol 0.1.2
0/0        3/3          0/0    0/0     0/0      !  escouade-plugins 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-updates 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-vectors 0.1.0
0/0        0/0          0/0    0/0     0/0      ?  escouade-voice 0.1.0
1/1        10/10        0/0    0/0     0/0      !  escouade-voice-local 0.1.0
```

C.2 Règles publiques : `semgrep`

`semgrep` 1.185.0, 149 règles des jeux publics `p/rust`, `p/typescript`, `p/javascript`, `p/security-audit`, `p/github-actions` et `p/secrets`, sur 1 633 fichiers hors tests : 162 résultats, dont 1 classé « erreur » et 6 « avertissement ». Aucun secret trouvé dans les sources ; aucune action GitHub référencée par une étiquette mobile.

Règle	Nombre	Lecture
AES-GCM sans longueur d'étiquette fixée (`store/server/utils/tokens.ts:35`)	1	Réel, mineur : le déchiffrement des jetons gardés par le store accepterait une étiquette d'authentification tronquée. Fixer `authTagLength` à 16. Serveur du store, hors du périmètre approfondi
Commande de masquage de secrets (`.github/workflows/release.yml:264`)	1	Le masquage est voulu. La ligne suivante pose les secrets Apple dans l'environnement du job : observation, SEC-18
Certificats non vérifiés (`src-tauri/src/testlaunch.rs:147`)	1	Voulu : sonde de disponibilité d'un serveur de développement local, sans proxy ; rien de ce qu'elle reçoit n'est lu
Caractères bidirectionnels (`store/server/utils/oauth.ts:90`)	4	Faux positif : c'est l'expression qui **retire** ces caractères des noms saisis. À écrire en séquences d'échappement pour lever le doute
Usage d'`unsafe`	68	Recoupe C.1
Dossier temporaire du système	73	Information. Pour l'essentiel dans les tests ; les usages hors tests n'ont pas été relus un à un
`current_exe`, arguments du processus	14	Information

L'outil a signalé 14 erreurs internes en lisant les workflows (deux règles n'ont pas pu s'appliquer à `ci.yml`, et `ci.yml` comme `release.yml` ne sont lus qu'en partie).

```
semgrep 1.185.0, règles publiques p/rust, p/typescript, p/javascript, p/security-audit, p/github-actions, p/secrets
Fichiers lus : 1633 ; résultats : 162 ; erreurs de lecture : 14

  73  INFO     rust.lang.security.temp-dir.temp-dir
        crates/escouade-acp/src/files.rs:177
        crates/escouade-acp/src/git.rs:415
        crates/escouade-acp/src/git.rs:563
        crates/escouade-acp/src/git.rs:837
        crates/escouade-base/src/testing.rs:29
        crates/escouade-engines/src/sidechain.rs:433
        … et 67 autres
  68  INFO     rust.lang.security.unsafe-usage.unsafe-usage
        crates/escouade-model/src/agentgroups.rs:222
        crates/escouade-model/src/agentgroups.rs:229
        crates/escouade-model/src/job.rs:63
        crates/escouade-model/src/job.rs:122
        crates/escouade-model/src/job.rs:144
        crates/escouade-model/src/job.rs:151
        … et 62 autres
   9  INFO     rust.lang.security.current-exe.current-exe
        crates/escouade-model/src/proc.rs:331
        crates/escouade-plugin-cli/src/cli.rs:370
        crates/escouade-plugins/src/lib.rs:742
        crates/escouade-plugins/src/lib.rs:760
        crates/escouade-voice-local/src/speech/piper.rs:187
        src-tauri/src/engine/terminal.rs:180
        … et 3 autres
   4  INFO     rust.lang.security.args.args
        crates/escouade-plugin-cli/src/main.rs:5
        src-tauri/src/bin/escouade-hook.rs:33
        src-tauri/src/bin/escouade-plugin.rs:7
        src-tauri/src/deeplink.rs:151
   4  WARNING  generic.unicode.security.bidi.contains-bidirectional-characters
        store/server/utils/oauth.ts:90
        store/server/utils/oauth.ts:90
        store/server/utils/oauth.ts:90
        store/server/utils/oauth.ts:90
   1  WARNING  yaml.github-actions.security.audit.unsafe-add-mask-workflow-command.unsafe-add-mask-workflow-command
        .github/workflows/release.yml:264
   1  INFO     rust.lang.security.args-os.args-os
        crates/escouade-plugin-cli/src/cli.rs:371
```

```
  1  WARNING  rust.lang.security.reqwest-accept-invalid.reqwest-accept-invalid
       src-tauri/src/testlaunch.rs:147
  1  ERROR    javascript.node-crypto.security.gcm-no-tag-length.gcm-no-tag-length
       store/server/utils/tokens.ts:35
```

C.3 Dépendances directes

```
Rust, application (src-tauri/Cargo.toml, 67 dépendances directes ; versions de Cargo.lock) :
  anyhow 1.0.104
  base64 0.13.1, 0.21.7, 0.22.1, 0.23.1
  chrono 0.4.45
  dirs 6.0.0, 7.0.0
  escouade-acp 0.1.6
  escouade-base 0.1.0
  escouade-engines 0.1.0
  escouade-forge 0.1.0
  escouade-mcp 0.1.0
  escouade-model 0.1.0
  escouade-plan 0.1.0
  escouade-plugin-cli 2.2.0
  escouade-plugin-kit 0.1.2
  escouade-plugin-protocol 0.1.2
  escouade-plugins 0.1.0
  escouade-updates 0.1.0
  escouade-vectors 0.1.0
  escouade-voice 0.1.0
  escouade-voice-local 0.1.0
  futures-util 0.3.34
  getrandom 0.2.17, 0.3.4, 0.4.3
  glob 0.3.4
  gtk 0.18.2
  http-body-util 0.1.5
  hyper 1.12.0
  hyper-util 0.1.21
  libc 0.2.191
  log 0.4.35
  notify 8.2.2
  objc2 0.6.5
  os_info 3.15.0
  parking_lot 0.12.5, 0.13.0
  pdf-extract 0.12.1
  portable-pty 0.9.0
  quick-xml 0.42.0
  reqwest 0.12.28, 0.13.5
  rmcp 3.9.0
  rusqlite 0.41.1
  rustls 0.23.46
  semver 1.0.28
  serde 1.0.229
  serde_json 1.0.154
  sha2 0.10.9, 0.11.1
  sys-locale 0.3.2
  tauri 2.12.0
  tauri-plugin-deep-link 2.6.0
  tauri-plugin-dialog 2.8.0
  tauri-plugin-notification 2.5.0
  tauri-plugin-opener 2.7.0
  tauri-plugin-process 2.4.0
  tauri-plugin-single-instance 2.5.5
  tauri-plugin-updater 2.13.1
  tauri-plugin-window-state 2.5.0
  tauri-winrt-notification 0.8.1
  tokio 1.54.1
  tokio-rustls 0.26.6
  tokio-tungstenite 0.30.0
  toml_edit 0.19.15, 0.20.2, 0.25.15+spec-1.1.0
  trash 5.3.0
  unicode-normalization 0.1.26
  uuid 1.29.2
  webkit2gtk 2.0.2
  webview2-com 0.39.1
  windows-core 0.62.2
  windows-sys 0.45.0, 0.52.0, 0.59.0, 0.60.2, 0.61.2
  yaml-edit 0.3.6
  zip 4.6.1, 9.5.0

npm, interface (package.json, 38 dépendances livrées ; versions de package-lock.json) :
  @codemirror/autocomplete 6.20.4
  @codemirror/commands 6.11.1
  @codemirror/lang-cpp 6.0.3
  @codemirror/lang-css 6.3.1
  @codemirror/lang-go 6.0.1
```

```
@codemirror/lang-html 6.4.12
@codemirror/lang-java 6.0.2
@codemirror/lang-javascript 6.2.5
@codemirror/lang-json 6.0.2
@codemirror/lang-markdown 6.5.2
@codemirror/lang-php 6.0.2
@codemirror/lang-python 6.2.1
@codemirror/lang-rust 6.0.2
@codemirror/lang-sql 6.10.0
@codemirror/lang-xml 6.1.0
@codemirror/lang-yaml 6.1.3
@codemirror/language 6.13.1
@codemirror/legacy-modes 6.5.5
@codemirror/merge 6.12.3
@codemirror/search 6.7.2
@codemirror/state 6.7.6
@codemirror/view 6.43.15
@fontsource-variable/inter 5.3.0
@fontsource/jetbrains-mono 5.3.0
@lezer/highlight 1.2.5
@tauri-apps/api 2.12.4
@tauri-apps/plugin-dialog 2.8.4
@tauri-apps/plugin-opener 2.7.2
@tauri-apps/plugin-process 2.4.0
@xterm/addon-fit 0.11.0
@xterm/addon-search 0.16.0
@xterm/addon-unicode11 0.9.0
@xterm/addon-web-links 0.12.0
@xterm/addon-webgl 0.19.0
@xterm/xterm 6.0.0
dompurify 3.4.17
marked 18.1.3
shiki 4.6.0
```

Annexe D. Tests réalisés

Tous les essais, y compris ceux qui n'ont rien trouvé. Les sources sont dans `docs/security/audit-2026-11/poc/`, les résultats bruts dans `docs/security/audit-2026-11/annexes/essais/`. « Tient » : l'attaque est repoussée. « Constat » : la faiblesse est montrée, et renvoie à sa fiche. « Limite » : la garde tient pour ce qu'elle nomme, et laisse passer le reste (section 6.10).

D.1 Boîte blanche : tests Rust d'attaque

Fichier `poc/whitebox/audit_2_2_0.rs`, 38 tests, compilés avec les tests de l'application et ignorés par défaut. Dernière exécution : 38 lancés, 38 réussis. Les tables les regroupent en 32 essais. Un test `holds_*` réussit quand l'attaque est repoussée ; un test `finding_*`, quand la faiblesse est bien là ; un test `limit_*` dit ce que la garde tient et imprime ce qu'elle laisse passer.

Id	Ce qui est tenté	Résultat
W-01	Lancement décrit par un plugin : fichier hors du dossier de l'agent par `..`, chemin relatif, UNC, préfixe `\\?\`, autre casse, lecteur relatif (14 chemins)	Tient
W-02	Même fichier à travers une jonction du dossier de l'agent	**Constat** SEC-06
W-03	Fichier écrit « à côté » du fichier privé d'une interface : autre dossier, `..`, autre nom	Tient ; un nom de script de même radical passe (`settings.cmd`) : SEC-06
W-04	Clôture d'un projet privé déclarée par un lancement tiers sans être appliquée	**Constat** SEC-02
W-05	Jeton d'Escouade placé dans un argument de harness	Tient
W-06	Variables d'environnement : 17 propres à Escouade, au chargeur ou au `PATH` ; puis 30 par lesquelles une commande en devient une autre	Tient : toutes refusées non déclarées ; déclarées, 28 refusées, 2 admises (`CLAUDE_CONFIG_DIR`, `ANTHROPIC_BASE_URL`), qui ne sont d'aucune famille refusée
W-07	`proc/spawn` : programme ou argument non déclaré, shell, chemin, dossier libre (11 cas) ; puis `git status` avec une configuration par variables	Tient : refusé, déclaré ou non, en majuscules ou en minuscules ; rien ne s'exécute
W-08	`fs/*` : `..`, chemin absolu, UNC, `\\?\`, `~`, flux NTFS, jonction sortante, dossier clôturé, autre casse, noms réservés de Windows	Tient (noms réservés : SEC-18)
W-09	`net/fetch` : 24 adresses piégées (schémas, sous-domaines trompeurs, identifiants dans l'adresse, adresses IP en toutes écritures) ; hôte déclaré qui résout vers la machine	Tient
W-10	26 adresses de la machine ou du réseau local, IPv4 et IPv6 ; réponse DNS mêlant adresses publiques et locales	Tient
W-11	Adresse IPv4 portée dans une adresse 6to4 ou Teredo	**Constat** SEC-09 (sans effet pratique connu)
W-12	Domaine de premier niveau entier déclaré comme hôte	**Constat** SEC-09
W-13	Sceau : 1 721 altérations d'un bit ou d'un caractère ; autre plugin, autre version ; clé révoquée ; liste non signée ; `version.json` altéré ou d'une autre version	Tient

Id	Ce qui est tenté	Résultat
W-14	Trame de harness : 256 Mio alloués sur la foi d'un en-tête de 29 octets	**Constat** SEC-08
W-15	Trame de harness : longueur supérieure à la mémoire	**Constat** SEC-08 : le processus s'arrête (`0xC0000409`)

Entrées générées (mutations d'entrées valides, 120 secondes par cible, sans instrumentation de couverture) :

Id	Cible	Entrées	Résultat
W-16	Lien `escouade://` (`deeplink::parse`)	55,5 millions	Aucune panique ; seul un identifiant bien formé est accepté
W-17	Manifeste de plugin (`manifest::check`)	4,6 millions, dont 217 430 acceptés	Aucune panique ; aucun manifeste accepté ne dépasse les droits d'un tiers
W-18	Archive du store (`store::read_archive`)	1,9 million	Aucune panique ; aucun chemin absolu ni sortant
W-19	Trames de harness (`read_frame`), longueurs bornées	37,8 millions	Aucune panique
W-20	Sceaux, listes de révocation, `version.json`	2,5 millions	Aucune panique ; aucune signature altérée acceptée pour d'autres octets
W-21	`.escouade.yml` et tickets partagés d'un dépôt (`projectfile::read` , `sharing::read`)	204 352	Aucune panique

Les gardes écrites comme des règles ou des listes, et les voies autour d'elles :

Id	Ce qui est tenté	Résultat
W-22	Variable refusée sous 27 autres graphies (casse, espace, signe, caractère nul, pleine chasse) ; déclarée dans une casse, posée dans une autre ; `GIT_AUTHOR_*` admis, `GIT_DIR` non	Tient
W-23	Dossier clôturé demandé par 13 graphies de Windows (nom court, point ou espace final, flux du répertoire, autre casse), en lecture et en écriture ; fichier d'un lancement par 7 graphies du dossier de l'agent	Tient
W-24	29 variables d'aucune famille refusée (`SSH_ASKPASS` , `LESSOPEN` , `RUSTC_WRAPPER` , `DOTNET_STARTUP_HOOKS` …)	Refusées non déclarées ; **limite** : les 29 passent une fois déclarées. La déclaration, affichée à l'installation, est alors le contrôle
W-25	Harness d'un tiers : 31 noms de shells et d'interpréteurs ; 49 autres lanceurs (`py` , `pythonw` , `pwsh-preview` , `npx` , `uvx` , `dotnet` …)	Les 31 refusés ; **limite** : les 49 passent (SEC-01)
W-26	Arguments du mode Bypass : 18 formes connues ; 12 autres écritures	Les 18 retirées ; **limite**, théorique : les 12 passent. Ce que chacune ouvre dépend du harness, qui n'a pas été lancé
W-27	« Ignorer la vérification » : onze écritures d'une exclusion du proxy (plage IPv4 et IPv6, joker, port, point final, crochets, tabulation) ; onze exclusions d'un autre hôte	Tient : les onze reconnues ; aucune des onze autres ne vaut à tort
W-28	La même, sur le fil : serveur local exclu par douze écritures, puis non exclu par cinq	Tient : exclu, il est joint directement et refusé pour son certificat, douze fois sur douze ; non exclu, il passe par le proxy

Id	Ce qui est tenté	Résultat
W-29	Vue Copilot CLI, demandée à son plugin comme le cœur la demande : ses refus d'écriture ; le mode sans demande, réglage éteint ; une discussion ; un agent clôturé	Tient pour ce que le cœur et le plugin écrivent : refus présents pour `.git`, `.copilot`, `.github/copilot`, `.github/hooks` ; réglage éteint, le cœur retire `--allow-all` ; une discussion refuse toute écriture et toute commande ; un agent clôturé n'obtient pas la vue. La tenue des refus dépend de Copilot CLI (non lancé)
W-30	`escouade-hook` appelé par un hook : 10 chemins d'installation qu'une ligne ne peut pas porter ; puis `cmd` lancé comme un hook dans un dossier qui tient un leurre du même nom	Tient : appelé par son nom, la variable qui interdit le dossier courant posée, son dossier en tête du `PATH`. Sur le poste, `cmd` lance le vrai programme avec la variable, et le leurre sans elle
W-31	Main d'un plugin selon son origine : dossier en développement, dossier copié, dépôt, sous un identifiant tiers et sous `dev.escouade.*` ; manifeste lu comme un tiers	Tient : toujours tiers ; l'identifiant réservé et les variables refusées sont refusés au manifeste
W-32	Harness décrit par un tiers : `install`, `updateCommand` et cinq variables de proxy et d'autorités dans `acp.removeEnv` ; ce que l'installation montre ; ce que l'application dit ; un lancement qui demande de retirer 28 variables, dont le proxy des réglages	Tient : le manifeste est refusé pour ses cinq variables ; l'installation montre les deux commandes et la variable retirée ; l'application dit que la commande vient du plugin ; le lancement garde les 18 variables d'une famille refusée ou données par le cœur, et le proxy. **Limite** : 10 variables d'aucune famille refusée se retirent

Tests du projet rejoués : 467 tests choisis par leur nom, dans les crates de l'application, autour de ces mêmes frontières (réponse entre agents, mode Bypass, certificats, portée des harness, variables des plugins, porte du serveur MCP, lecture des quotas et de la connexion de Claude Code, relance et attache d'un harness, ce qu'un harness tiers dit sans le lancer). Résultat : 467 réussis. Les 37 tests de la vue des plugins (`src/components/plugins/plugins.test.ts`) passent aussi.

D.2 Boîte noire : l'application qui tourne

Instance isolée : binaire de développement construit à l'étiquette `v2.2.0`, dossier de données et profil jetables, faux harness, ports réservés. Fichiers `poc/blackbox/*.spec.ts`, 16 scénarios. Dernière exécution : 16 réussis.

Id	Ce qui est tenté	Résultat
B-01	Ports ouverts par l'application et ses processus ; connexion depuis l'adresse réseau du poste	Tient : `127.0.0.1` seulement
B-02	Serveur MCP : 41 requêtes forgées (sans jeton, mauvais jeton, jeton ailleurs que dans son en-tête, 7 `Host` forgés, 5 `Origin`, URI absolue, pré-requête CORS, 8 chemins avec et sans jeton, corps invalide ou démesuré)	Tient : 401, 403 ou 404 selon le cas, aucun en-tête CORS, le jeton dans aucun journal
B-03	80 connexions muettes, puis une requête légitime	Servie après 8,5 s : SEC-17
B-04	Agent contre agent par le serveur MCP : répondre à sa propre demande, « toujours », joindre un fichier hors de son dossier, passer en `bypassPermissions`, s'arrêter, s'écrire ; puis répondre à la demande d'un autre, par quatre voies	Tient : tout est refusé ; c'est l'utilisateur qui répond
B-05	Noms d'outils inconnus de 256 Kio envoyés avec un jeton : 1 000 requêtes, puis 1 800 par neuf formes détournées	Tient : toutes refusées (400, 413 ou 415), mémoire de l'application stable ; sans jeton : 401
B-06	32 charges dans un message d'agent : scripts, gestionnaires d'événements, images et cadres distants, liens `javascript:`, `data:`, `file:`, UNC, formulaires, styles, `meta`, `base`, mXSS	Tient : aucun script, aucune requête, aucun élément actif

Id	Ce qui est tenté	Résultat
B-07	Script, `eval`, `fetch`, WebSocket, image, cadre, média injectés directement dans la page	Tient : 15 violations bloquées ; un style en ligne s'applique : SEC-05
B-08	Image d'un chemin local : dans le projet, hors du projet, fichier qui n'est pas une image, fichier système, partage	Seules les images sont lues (SEC-18)
B-09	Plugin malveillant, 69 demandes hors de ses permissions : réseau (15), fichiers et descripteurs bruts (17), programmes et variables (24), secrets (6), méthodes du cœur (7)	Tient : 63 refusées ; passent les 6 que son manifeste permet, dont 3 variables déclarées (parmi elles `SSH_ASKPASS` : W-24). Son secret est masqué dans le journal
B-10	Plugin malveillant : prendre de la mémoire sans fin	Tient : instance arrêtée, plus relancée après 4 arrêts ; l'application passe de 18 Mio à 153 Mio au plus, puis revient à 43 Mio
B-11	Plugin malveillant : tour de calcul sans fin	Tient : arrêté à 2 s
B-12	Plugin malveillant : écrire sans fin	L'application tient, avec un pic de 312 Mio : SEC-11
B-13	Canal de mise à jour contre un serveur local : 14 cas (signature absente, mal formée, d'une autre clé, d'autres octets ; version plus ancienne, égale, bêta ; manifeste invalide ; installateur par `file://` ou par un partage)	Tient : rien ne s'installe, rien d'ancien n'est proposé
B-14	Dossier en développement qui déclare `GIT_CONFIG_*`, sous un identifiant tiers puis `dev.escouade.*` ; et sous `dev.escouade.*` sans variable refusée	Tient : les trois dossiers sont refusés à la lecture du manifeste
B-15	« Versions vérifiées » allumé : installer le plugin d'essai depuis un dossier, puis en développement	Les deux s'installent : **constat** SEC-01
B-16	« Tout mettre à jour » avec une mise à jour que personne n'a relue et qui ne demande rien de plus	Tient : aucune fenêtre commune ne la porte ; elle vient dans sa propre fenêtre, avec son avertissement ; fermée sans accord, rien n'est installé

Le plugin malveillant seul, dans l'hôte des plugins de cette version (`escouade-plugin dev`) : 69 demandes, 63 refusées, 6 permises par son manifeste.

D.3 Store et site, en lecture seule

Une douzaine de requêtes GET, aucun compte, aucune écriture. Le store a été lu le 12 novembre 2026, le site le 14.

Id	Ce qui est lu	Résultat
N-01	En-têtes de la page d'accueil du store	Complets (HSTS, politique de contenu à nonce, `nosniff`, `DENY`) ; `X-Powered-By` présent
N-02	`/api/v1/catalog.json`	Public par conception, mêmes en-têtes. 28 plugins, tous de première main (`dev.escouade.*`) ; 112 versions, dont 89 vérifiées, aucune hors de cet espace (SEC-03)
N-03	`/api/v1/revocations.json`	Liste nº 1, vide
N-04	Requête avec une origine étrangère	Aucun en-tête CORS
N-05	Accès en `http`	Redirigé vers `https` (store et site)

Id	Ce qui est lu	Résultat
N-06	TLS 1.0 et 1.1	Refusés
N-07	En-têtes du site de l'éditeur	**Constat** SEC-16

D.4 Le produit publié

P-01 à P-04 : section 6.11. Les fichiers de la version publiée, contrôlés sans être lancés.

D.5 Ce qui n'a pas été essayé

- Les harness réels et leurs bacs à sable : donc le sens des arguments de W-26 et la tenue des refus de W-29.
- macOS et Linux : donc la validité de la signature et de la notarisation du paquet macOS, lues dans le paquet et dans le journal du job.
- L'application installée depuis l'installateur publié (section 2, « Limites »).
- L'installation d'un plugin tiers qui décrit un harness, jusqu'au lancement de son programme (lu dans le code ; W-25 et W-32 éprouvent ses contrôles).
- Le lien `escouade://` transmis à une instance déjà ouverte (seul le lecteur du lien a été essayé).
- Le serveur de hooks en boîte noire (lu dans le code).
- Toute écriture sur le store, toute attaque d'un service tiers.

Annexe E. Guide de durcissement pour un poste d'entreprise

Les réglages à poser sur un poste d'entreprise qui tient du code sensible, tels que la version 2.2.0 les nomme. La colonne « Défaut » dit l'état d'une installation neuve de la 2.2.0 ; une installation mise à jour depuis une version antérieure garde ses anciens choix, signalés quand ils diffèrent.

Sujet	Réglage	Défaut	Valeur conseillée	Où
Statistiques d'usage	Question du premier démarrage	Non répondu : rien ne part	**Refuser**	Premier démarrage ; Réglages › Général
Mode des agents	« Mode par défaut des agents », mode de chaque compte de harness, mode du Kanban, mode de chaque agent	Question du premier démarrage, « Édits auto » recommandé. Installation mise à jour : `auto`	**« Demander »** (`default`) ou « Plan »	Réglages › Agents ; Réglages › Comptes ; Réglages du projet › Kanban ; menu du mode sous la conversation
Mode Bypass	« Autoriser le mode Bypass »	Éteint. Installation mise à jour qui s'en servait : allumé	**Éteint**	Réglages › Agents
Pilotage par les agents	« Les agents peuvent piloter Escouade »	Éteint	**Éteint**	Réglages › Agents
Réponse entre agents	« Les agents peuvent répondre aux questions et aux permissions des autres agents »	« Personne »	**« Personne »**	Réglages › Agents
Remote Control	Global, par projet, par agent	Éteint	**Éteint** partout	Réglages › Agents ; Réglages du projet › Confidentialité
Plugins tiers	« N'installer que des versions vérifiées »	Allumé. Installation mise à jour : **éteint**	**Allumé**, et aucun plugin tiers, ni d'un dossier ni d'un dépôt : le réglage ne juge que le store (SEC-01)	Réglages › Plugins
Plugins en développement	Chargement d'un dossier	Aucun	**Aucun** : c'est du code que personne n'a relu	Plugins
Mises à jour des plugins	« Tout mettre à jour »	—	Sans plugin tiers, sans objet. Une version non vérifiée a toujours sa propre fenêtre : lire ce qu'elle dit avant d'accepter	Plugins › Mises à jour
Certificats	« Autorités de certification supplémentaires »	Vide	Le certificat racine du proxy de l'entreprise, si le magasin du système ne le porte pas	Réglages › Avancé › Réseau
Certificats	« Ignorer la vérification des certificats TLS »	Décoché	**Décoché** : le fichier d'autorités la remplace	Réglages › Avancé › Réseau
Proxy	Adresse du proxy	Vide	Le proxy de l'entreprise, **sans mot de passe dans l'adresse**	Réglages › Avancé › Réseau

Sujet	Réglage	Défaut	Valeur conseillée	Où
Voix	Services de voix en ligne	Aucun	**Aucun** ; la dictée locale n'envoie rien	Réglages › Voix
Comptes	Hébergeurs de code, sources de tickets	Aucun	Seulement ceux dont le poste a besoin, avec un jeton à droits minimaux et à durée courte	Réglages › Comptes
Mises à jour d'Escouade	Canal bêta	Éteint	**Éteint**	Réglages › Général
Mises à jour de Claude Code	« Mettre à jour Claude Code automatiquement »	Allumé	Selon la politique de l'entreprise sur les versions des CLI	Réglages › Comptes (avancé)
Projets privés	« Projet privé »	Éteint	Ne pas s'y fier pour séparer des projets qui ne doivent pas se voir (SEC-02)	—
Fichiers joints	« Effacer les copies après »	Jamais	Une durée courte	Réglages › Avancé

Au pare-feu ou au proxy. Autoriser, selon la politique de l'entreprise : le fournisseur de modèles du harness retenu (flux R5, et R4 pour Claude Code, qui lit lui-même ses quotas), l'hébergeur de code des projets (R3, R8). Peuvent être bloqués sans gêner le travail : `store.escouade.dev` (R2), l'hôte des statistiques (R7), les services de voix (R10), `huggingface.co` (R11). Bloquer R1 (les mises à jour) impose de déployer les nouvelles versions par un autre moyen : à décider avec l'éditeur, car les correctifs de sécurité passent par là.

Sur le poste. Chiffrement du disque. Un compte du système par ensemble de projets à séparer, si le poste en tient plusieurs. Les CLI des harness installés et configurés selon la politique de l'entreprise : ce sont eux qui parlent aux fournisseurs de modèles.

À surveiller. Le fichier `app.log` du dossier de données ; le journal d'activité du serveur MCP s'il est allumé ; la page Plugins, qui liste ce qui est installé et d'où.

Avant de déployer. Exiger la version 2.2.0 ou une version plus récente. Sur une installation mise à jour depuis une version antérieure, relire quatre réglages, dont elle garde les anciens choix : mode des agents, mode Bypass, versions vérifiées, réponse entre agents.

Annexe F. Reproduire l'audit

Depuis la racine du dépôt, au commit qui porte ce dossier : les essais sont écrits pour le code de la 2.2.0. Les ports sont donnés en variables, jamais écrits dans les fichiers.

```
# Outils
cargo install cargo-audit cargo-deny cargo-geiger --locked
(cd src-tauri && cargo audit) ; (cd plugins && cargo audit)
cargo deny --manifest-path src-tauri/Cargo.toml --config deny.toml check
cargo deny --manifest-path plugins/Cargo.toml --config deny.toml check
npm audit ; npm audit --omit=dev
(cd store && npm audit --omit=dev) ; (cd website && npm audit --omit=dev)
(cd src-tauri && cargo geiger --output-format Ascii)
semgrep scan --metrics=off --config p/rust --config p/typescript --config p/javascript \
  --config p/security-audit --config p/github-actions --config p/secrets \
  --exclude "*.test.ts" --exclude "*_tests.rs" --exclude tests --json --output <sortie> \
  src-tauri/src crates src store/server store/app plugins .github/workflows scripts

# Boîte blanche (prérequis du dépôt : sh scripts/escouade-js.sh ; sh scripts/plugins-wasm.sh)
cd src-tauri
AUDIT_FUZZ_SECONDS=120 cargo nextest run --lib -p escouade \
  --run-ignored ignored-only -E 'test(audit_2_2_0)' --no-capture

# Boîte noire (prérequis : npx tauri build --debug --no-bundle)
ESCOUADE_E2E_CDP_PORT=<port> AUDIT_PORT_BASE=<port> AUDIT_OUT=<dossier> \
  npx playwright test -c docs/security/audit-2026-11/poc/blackbox

# Le plugin malveillant, seul, dans l'hôte des plugins
src-tauri/target/debug/escouade-plugin dev docs/security/audit-2026-11/poc/malicious-plugin

# La version publiée : ses fichiers dans un dossier neuf, hors du dépôt, jamais lancés
gh release download v2.2.0 --repo <dépôt de l'éditeur> --dir <dossier>
node docs/security/audit-2026-11/tools/verify-update-signature.mjs \
  src-tauri/tauri.conf.json --manifest <dossier>/latest.json <dossier>
node docs/security/audit-2026-11/tools/inspect-macos-package.mjs <app.tar.gz> <dmg>
node docs/security/audit-2026-11/tools/verify-provenance.mjs <dossier> <dépôt de l'éditeur> <commit de
l'étiquette>

# Les annexes, à partir des sorties brutes, puis ce rapport
node docs/security/audit-2026-11/tools/annexes.mjs <dossier des sorties brutes>
node docs/security/audit-2026-11/tools/references.mjs check
node docs/security/audit-2026-11/build.mjs
```

Les essais n'emploient aucune charge utile réutilisable : chaque « exploitation » écrit un fichier témoin dans un dossier de l'essai, ou mesure la mémoire de l'instance de test. `cargo geiger` nettoie les binaires de développement : reconstruire l'application avant la boîte noire.